Machine Learning in Finance: Final Project

Machine Learning in NBA Over/Under Betting

Business 35137 | Chicago Booth

Research Question: Can machine learning models beat the Vegas total when predicting NBA game points, and under what conditions do selective bets outperform a flat-bet strategy?

Key Idea: Vegas lines are remarkably efficient — they incorporate enormous amounts of information. Our models must find systematic patterns that the market misses. We focus on:

- **Model performance vs the Vegas line** (predicting game totals)
- **Selective betting** when the model strongly disagrees with the line
- **Robustness checks** and statistical significance

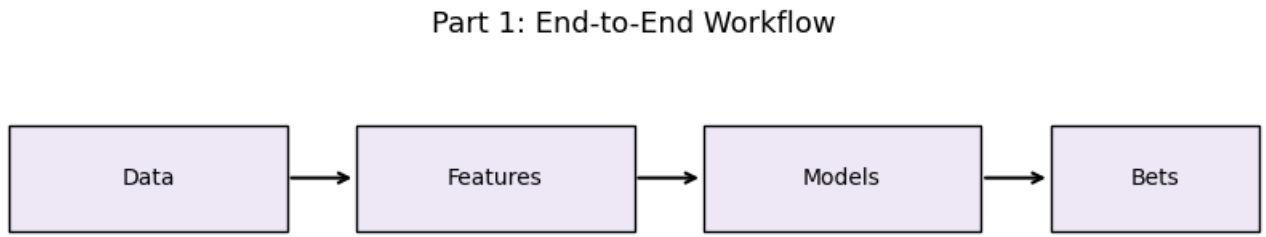
Data: 23,118 NBA games (2008–2025) with betting lines and box score statistics.

Part 1: Setup and Imports

We set up a reproducible workflow with shared plotting styles and consistent preprocessing. The diagram below shows the end-to-end flow we follow throughout the notebook, from raw data to betting decisions.

In [75]:
```
# Figure: Analysis pipeline overview
import matplotlib.patches as patches

fig, ax = plt.subplots(figsize=(9, 2.2))
ax.axis('off')

boxes = [
    (0.05, 0.3, 0.2, 0.4, 'Data'),
    (0.30, 0.3, 0.2, 0.4, 'Features'),
    (0.55, 0.3, 0.2, 0.4, 'Models'),
    (0.80, 0.3, 0.15, 0.4, 'Bets'),
 ]
for x, y, w, h, label in boxes:
    rect = patches.Rectangle((x, y), w, h, facecolor='#EDE7F6', edgecolor='black')
    ax.add_patch(rect)
    ax.text(x + w/2, y + h/2, label, ha='center', va='center', fontsize=10)

for i in range(len(boxes) - 1):
    x1 = boxes[i][0] + boxes[i][2]
    x2 = boxes[i + 1][0]
    ax.annotate('', xy=(x2, 0.5), xytext=(x1, 0.5),
                arrowprops=dict(arrowstyle='->', lw=1.5))

ax.set_title('Part 1: End-to-End Workflow')
```

```
plt.tight_layout()
plt.savefig('Images/part1_pipeline.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [76]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import warnings
import time
import requests
from math import erf, sqrt
warnings.filterwarnings('ignore')

from sklearn.linear_model import Ridge, Lasso, ElasticNet, LinearRegression
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from nba_api.stats.endpoints import leaguegamelog

# Consistent plot style
plt.rcParams.update({
    'figure.facecolor': 'white',
    'axes.facecolor': 'white',
    'axes.grid': True,
    'grid.alpha': 0.3,
    'font.size': 11,
    'axes.titlesize': 13,
    'axes.labelsize': 11,
    'figure.titlesize': 15,
})

COLORS = {
    'primary': '#2196F3',      # blue  (Lasso / sparse)
    'secondary': '#FF9800',    # orange (RBF+Ridge / complex)
    'profit': '#4CAF50',       # green
    'loss': '#F44336',         # red
    'neutral': '#9E9E9E',      # gray
    'highlight': '#7C4DFF',    # purple
    'breakeven': '#F44336',    # red dashed
    'vegas': '#607D8B',        # blue-gray
}
```

```
os.makedirs('Images', exist_ok=True)
print("Setup complete.")
```

```
Setup complete.
```

Part 2: Data

We combine two sources and then fill gaps:

1. **Betting data** (2008–2025): Vegas totals, spreads, and actual scores (23,118 games)
2. **NBA game data** (2008–2023): Box score statistics from a historical database

To fix missing seasons (notably 2012–2013), we pull team game logs from the NBA Stats API and merge them into the game dataset. We also use the API for 2024–2025 in the same format.

Why coverage differs in 2020: the NBA season was shortened and rescheduled due to COVID. Betting data still records every listed game, while the box-score dataset can miss or drop some rows after team-code standardization and duplicate removal. The gap is a reminder that the two sources are curated differently even when they refer to the same season.

Line note: The betting file provides a single total and spread per game (no explicit open/close fields). We treat it as the market line in the dataset; separating opening vs closing would require a different data source.

In [77]:
```
# Team abbreviation mappings
TEAM_MAP_BETTING = {'gs': 'GSW', 'no': 'NOP', 'ny': 'NYK', 'sa': 'SAS', 'utah': 'UTA', 'wsh': 'WAS'}
TEAM_MAP_HISTORICAL = {'NJN': 'BKN', 'NOH': 'NOP', 'NOK': 'NOP', 'NOJ': 'NOP',
                       'SEA': 'OKC', 'CHN': 'CHA', 'CHH': 'CHA', 'VAN': 'MEM'}

def standardize_team(abbr, source='betting'):
    if source == 'betting':
        return TEAM_MAP_BETTING.get(abbr, abbr.upper())
    if source == 'historical':
        return TEAM_MAP_HISTORICAL.get(abbr, abbr)
    return abbr


def load_api_game_data(seasons, cache_dir='Data/Game Data'):
    all_rows = []
    for season in seasons:
        cache_path = os.path.join(cache_dir, f"game_{season.replace('-', '_')}_api.csv")
        if os.path.exists(cache_path):
            season_df = pd.read_csv(cache_path)
            all_rows.append(season_df)
            continue
        dfs = []
        for stype in ['Regular Season', 'Playoffs']:
            log = leaguegamelog.LeagueGameLog(season=season, season_type_all_star=stype)
            df = log.get_data_frames()[0]
            dfs.append(df)
            time.sleep(1.0)
        season_df = pd.concat(dfs, ignore_index=True)
        season_df.to_csv(cache_path, index=False)
        all_rows.append(season_df)
```

```python
    return pd.concat(all_rows, ignore_index=True) if all_rows else pd.DataFrame()


def api_team_logs_to_games(api_df):
    if api_df.empty:
        return pd.DataFrame()
    api_df = api_df.copy()
    api_df['GAME_DATE'] = pd.to_datetime(api_df['GAME_DATE'], format='mixed', errors='coerce')
    api_df = api_df.dropna(subset=['GAME_DATE'])
    api_df['is_home'] = api_df['MATCHUP'].str.contains(r'vs\.').astype(int)
    home_api = api_df[api_df['is_home'] == 1].copy()
    away_api = api_df[api_df['is_home'] == 0].copy()
    merged = home_api.merge(away_api, on='GAME_ID', suffixes=('_home', '_away'))

    api_map = {'PTS': 'pts', 'FGM': 'fgm', 'FGA': 'fga', 'FG_PCT': 'fg_pct',
               'FG3M': 'fg3m', 'FG3A': 'fg3a', 'FG3_PCT': 'fg3_pct',
               'FTM': 'ftm', 'FTA': 'fta', 'FT_PCT': 'ft_pct',
               'OREB': 'oreb', 'DREB': 'dreb', 'REB': 'reb',
               'AST': 'ast', 'STL': 'stl', 'BLK': 'blk', 'TOV': 'tov', 'PF': 'pf'}
    api_rename = {'GAME_ID': 'game_id', 'GAME_DATE_home': 'game_date',
                  'TEAM_ABBREVIATION_home': 'home_team', 'TEAM_ABBREVIATION_away': 'away_team'}
    for ac, lc in api_map.items():
        api_rename[f'{ac}_home'] = f'{lc}_home'
        api_rename[f'{ac}_away'] = f'{lc}_away'
    return merged[list(api_rename.keys())].rename(columns=api_rename)


def load_game_data(api_seasons=None):
    """Load and merge box score data: game.csv (2008-2023) + API data (2012-13 + 2024-2025)."""
    # Historical
    game_hist = pd.read_csv('Data/Game Data/game.csv')
    game_hist['game_date'] = pd.to_datetime(game_hist['game_date'])
    game_hist = game_hist[
        (game_hist['season_id'].astype(str).str[1:].astype(int) >= 2007) &
        (game_hist['season_type'].isin(['Regular Season', 'Playoffs']))
    ].copy()
    for col in ['team_abbreviation_home', 'team_abbreviation_away']:
        game_hist[col] = game_hist[col].map(lambda x: standardize_team(x, 'historical'))

    stats = ['pts', 'fgm', 'fga', 'fg_pct', 'fg3m', 'fg3a', 'fg3_pct',
             'ftm', 'fta', 'ft_pct', 'oreb', 'dreb', 'reb', 'ast', 'stl', 'blk', 'tov', 'pf']
    rename = {'game_id': 'game_id', 'game_date': 'game_date',
              'team_abbreviation_home': 'home_team', 'team_abbreviation_away': 'away_team'}
    for s in stats:
        rename[f'{s}_home'] = f'{s}_home'
        rename[f'{s}_away'] = f'{s}_away'
    game_hist = game_hist[list(rename.keys())].rename(columns=rename)

    # API data (CSV for 2024-2025 + fetch for 2012-2013)
    api_csv = pd.read_csv('Data/Game Data/game_2024_2025.csv')
    if api_seasons is None:
        api_seasons = ['2012-13']
    api_extra = load_api_game_data(api_seasons) if api_seasons else pd.DataFrame()
    api_all = pd.concat([api_csv, api_extra], ignore_index=True)
    api_games = api_team_logs_to_games(api_all)

    all_games = pd.concat([game_hist, api_games], ignore_index=True)
```

```python
    all_games['game_key'] = all_games['game_date'].dt.strftime('%Y%m%d') + '_' + all_games['home_team'] + '_' + all_games['away_team']
    all_games = all_games.sort_values('game_date').drop_duplicates('game_key').drop(columns=['game_key'])
    print(f"Game data: {len(all_games):,} games ({len(game_hist):,} historical + {len(api_games):,} API)")
    return all_games

def load_betting_data(filepath='Data/Betting Data/nba_2008-2025.csv'):
    df = pd.read_csv(filepath)
    df['date'] = pd.to_datetime(df['date'])
    df = df.sort_values('date').reset_index(drop=True)
    df['actual_total'] = df['score_away'] + df['score_home']
    df['over_hit'] = (df['actual_total'] > df['total']).astype(int)
    print(f"Betting data: {len(df):,} games ({df['date'].min().date()} to {df['date'].max().date()})")
    return df


def compute_rolling_features(game_data, windows=[5, 10]):
    """Compute lagged rolling stats. All use shift(1) to prevent lookahead."""
    home = game_data[['game_date', 'home_team', 'pts_home', 'pts_away',
        'fg_pct_home', 'fg3_pct_home', 'ft_pct_home',
        'oreb_home', 'dreb_home', 'reb_home', 'ast_home', 'stl_home', 'blk_home', 'tov_home']].copy()
    home.columns = ['game_date', 'team', 'pts_scored', 'pts_allowed',
        'fg_pct', 'fg3_pct', 'ft_pct', 'oreb', 'dreb', 'reb', 'ast', 'stl', 'blk', 'tov']
    home['is_home'] = 1
    home['total'] = home['pts_scored'] + home['pts_allowed']
    home['win'] = (home['pts_scored'] > home['pts_allowed']).astype(int)

    away = game_data[['game_date', 'away_team', 'pts_away', 'pts_home',
        'fg_pct_away', 'fg3_pct_away', 'ft_pct_away',
        'oreb_away', 'dreb_away', 'reb_away', 'ast_away', 'stl_away', 'blk_away', 'tov_away']].copy()
    away.columns = ['game_date', 'team', 'pts_scored', 'pts_allowed',
        'fg_pct', 'fg3_pct', 'ft_pct', 'oreb', 'dreb', 'reb', 'ast', 'stl', 'blk', 'tov']
    away['is_home'] = 0
    away['total'] = away['pts_scored'] + away['pts_allowed']
    away['win'] = (away['pts_scored'] > away['pts_allowed']).astype(int)

    all_tg = pd.concat([home, away]).sort_values('game_date')
    stat_cols = ['pts_scored', 'pts_allowed', 'fg_pct', 'fg3_pct', 'ft_pct',
                 'oreb', 'dreb', 'reb', 'ast', 'stl', 'blk', 'tov', 'total', 'win']

    results = {}
    for team in all_tg['team'].unique():
        tg = all_tg[all_tg['team'] == team].copy().sort_values('game_date')
        tg['days_rest'] = tg['game_date'].diff().dt.days
        tg['back_to_back'] = (tg['days_rest'] == 1).astype(int)
        for w in windows:
            for col in stat_cols:
                tg[f'{col}_L{w}'] = tg[col].shift(1).rolling(w, min_periods=1).mean()
            tg[f'days_rest_L{w}'] = tg['days_rest'].shift(1).rolling(w, min_periods=1).mean()
        results[team] = tg
    return pd.concat(results.values())


def create_feature_matrix(betting_df, game_data, windows=[5, 10]):
    """Join betting data with rolling game stats -> ~80 features."""
    print("Computing rolling features...")
    rolling = compute_rolling_features(game_data, windows)
    roll_cols = [c for c in rolling.columns if '_L' in c]
```

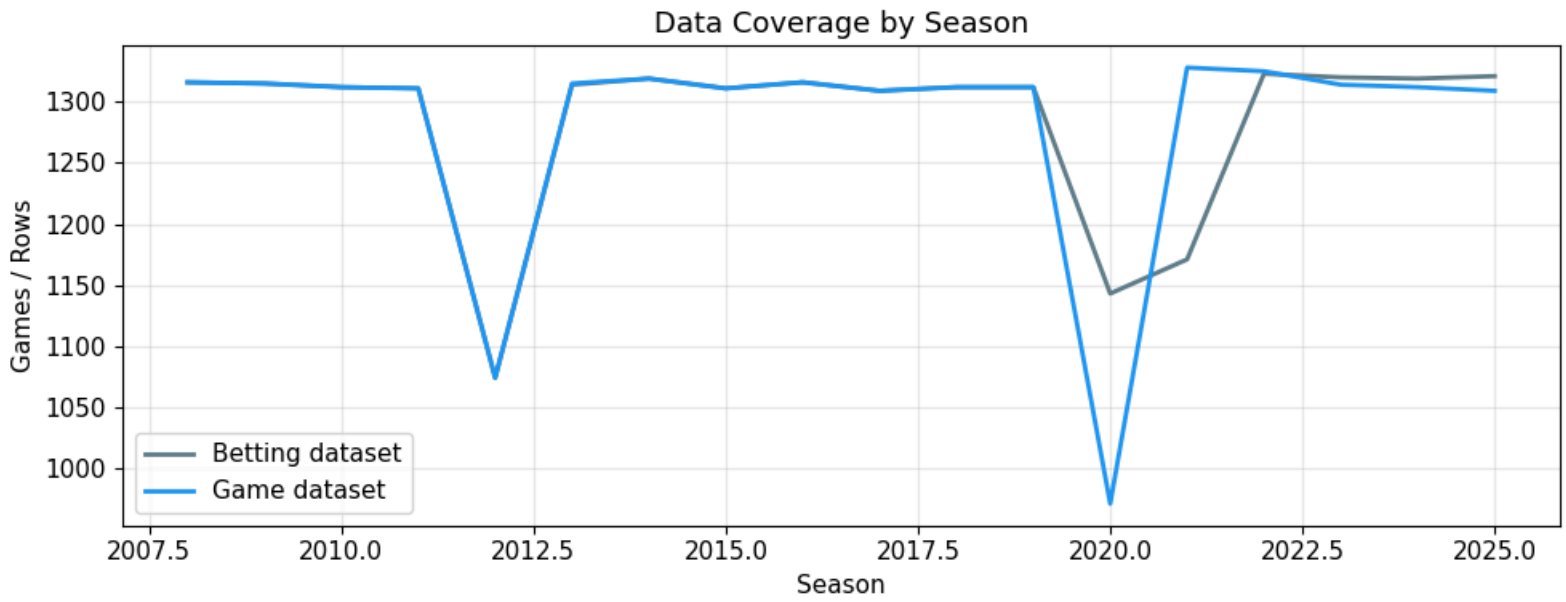
```python
    df = betting_df.copy()
    df['home_std'] = df['home'].map(lambda x: standardize_team(x, 'betting'))
    df['away_std'] = df['away'].map(lambda x: standardize_team(x, 'betting'))
    features = df[['date', 'home', 'away', 'home_std', 'away_std',
                   'total', 'spread', 'actual_total', 'over_hit', 'season']].copy()

    home_roll = rolling[rolling['is_home'] == 1][['game_date', 'team'] + roll_cols].copy()
    home_roll.columns = ['date', 'home_std'] + ['H_' + c for c in roll_cols]
    features = features.merge(home_roll, on=['date', 'home_std'], how='left')

    away_roll = rolling[rolling['is_home'] == 0][['game_date', 'team'] + roll_cols].copy()
    away_roll.columns = ['date', 'away_std'] + ['A_' + c for c in roll_cols]
    features = features.merge(away_roll, on=['date', 'away_std'], how='left')

    for w in windows:
        features[f'combined_pts_L{w}'] = features[f'H_pts_scored_L{w}'] + features[f'A_pts_scored_L{w}']
        features[f'combined_allowed_L{w}'] = features[f'H_pts_allowed_L{w}'] + features[f'A_pts_allowed_L{w}']
        features[f'expected_total_L{w}'] = (
            features[f'H_pts_scored_L{w}'] + features[f'A_pts_scored_L{w}'] +
            features[f'H_pts_allowed_L{w}'] + features[f'A_pts_allowed_L{w}']) / 2
        features[f'combined_reb_L{w}'] = features[f'H_reb_L{w}'] + features[f'A_reb_L{w}']
        features[f'combined_ast_L{w}'] = features[f'H_ast_L{w}'] + features[f'A_ast_L{w}']
        features[f'combined_tov_L{w}'] = features[f'H_tov_L{w}'] + features[f'A_tov_L{w}']
        features[f'combined_fg_pct_L{w}'] = (features[f'H_fg_pct_L{w}'] + features[f'A_fg_pct_L{w}']) / 2
        features[f'combined_fg3_pct_L{w}'] = (features[f'H_fg3_pct_L{w}'] + features[f'A_fg3_pct_L{w}']) / 2
        features[f'rest_diff_L{w}'] = features[f'H_days_rest_L{w}'] - features[f'A_days_rest_L{w}']

    features['vegas_total'] = features['total']
    features['vegas_spread'] = features['spread']
    features = features.drop(columns=['home_std', 'away_std'], errors='ignore')

    n_before = len(features)
    features = features.dropna()
    print(f"Feature matrix: {len(features):,} games (dropped {n_before - len(features):,} with insufficient history)")
    return features


def get_feature_columns(df):
    exclude = {'date', 'home', 'away', 'total', 'actual_total', 'over_hit', 'season', 'spread',
               'score_home', 'score_away', 'regular', 'playoffs',
               'whos_favored', 'moneyline_away', 'moneyline_home',
               'h2_spread', 'h2_total', 'id_spread', 'id_total',
               'q1_away', 'q2_away', 'q3_away', 'q4_away', 'ot_away',
               'q1_home', 'q2_home', 'q3_home', 'q4_home', 'ot_home'}
    return [c for c in df.columns if c not in exclude]


print("=" * 70)
print("DATA PIPELINE")
print("=" * 70)
game_data = load_game_data()
betting_data = load_betting_data()
```

```
======================================================================
DATA PIPELINE
======================================================================
Game data: 23,081 games (19,230 historical + 3,936 API)
Betting data: 23,118 games (2007-10-30 to 2025-06-22)
Betting data: 23,118 games (2007-10-30 to 2025-06-22)
```

In [78]:

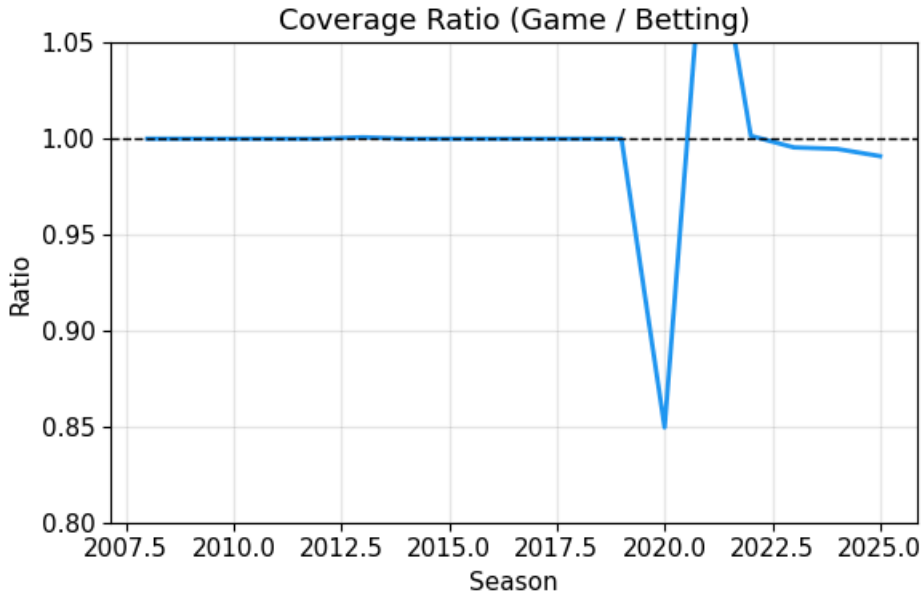
```python
# Data coverage by season
betting_counts = betting_data.groupby('season').size().reset_index(name='betting_games')
game_data['season'] = game_data['game_date'].dt.year + (game_data['game_date'].dt.month >= 7)
game_counts = game_data.groupby('season').size().reset_index(name='game_rows')

cov = betting_counts.merge(game_counts, on='season', how='left')
print(cov.tail(10).to_string(index=False))

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(cov['season'], cov['betting_games'], label='Betting dataset', color=COLORS['vegas'], lw=2)
ax.plot(cov['season'], cov['game_rows'], label='Game dataset', color=COLORS['primary'], lw=2)
ax.set_title('Data Coverage by Season')
ax.set_xlabel('Season'); ax.set_ylabel('Games / Rows')
ax.legend()
plt.tight_layout()
plt.savefig('Images/data_coverage.png', dpi=150, bbox_inches='tight')
plt.show()
```

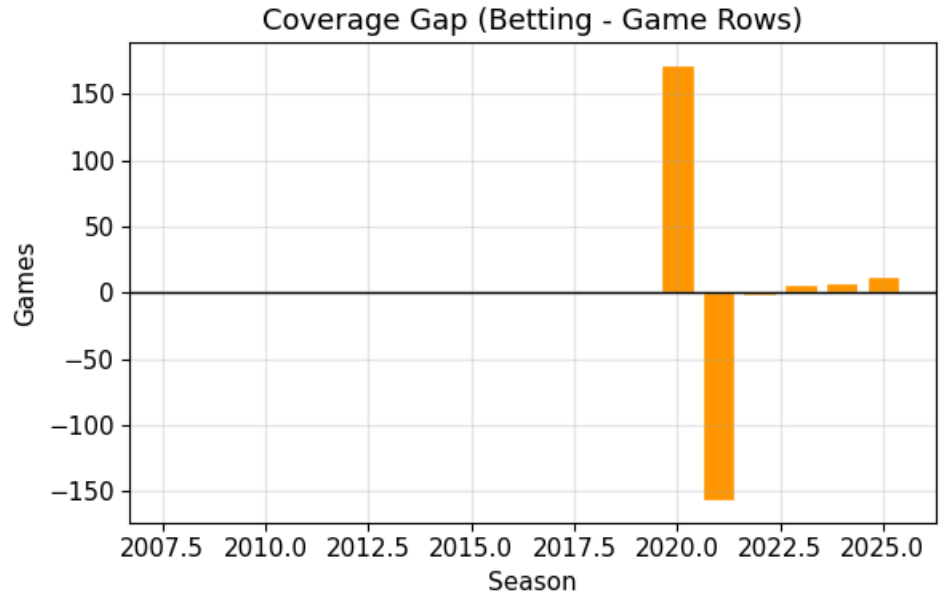
```
season  betting_games  game_rows
  2016           1316       1316
  2017           1309       1309
  2018           1312       1312
  2019           1312       1312
  2020           1143        971
  2021           1171       1328
  2022           1323       1325
  2023           1320       1314
  2024           1319       1312
  2025           1321       1309
```



In [79]:

```
# Figure: Coverage ratio and gap
cov = cov.copy()
cov['coverage_ratio'] = cov['game_rows'] / cov['betting_games']
cov['gap'] = cov['betting_games'] - cov['game_rows']

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

ax = axes[0]
ax.plot(cov['season'], cov['coverage_ratio'], color=COLORS['primary'], lw=2)
ax.axhline(1.0, color='black', lw=1, ls='--')
ax.set_title('Coverage Ratio (Game / Betting)')
ax.set_xlabel('Season'); ax.set_ylabel('Ratio')
ax.set_ylim(0.8, 1.05)

ax = axes[1]
ax.bar(cov['season'], cov['gap'], color=COLORS['secondary'], edgecolor='white')
ax.axhline(0, color='black', lw=1)
ax.set_title('Coverage Gap (Betting - Game Rows)')
ax.set_xlabel('Season'); ax.set_ylabel('Games')

plt.tight_layout()
plt.savefig('Images/data_coverage_gap.png', dpi=150, bbox_inches='tight')
plt.show()
```





Observations:

- The 2012–2013 drop is now largely repaired after pulling game logs from the NBA Stats API.
- 2020 shows a visible dip because the season itself was shorter; any residual mismatch reflects source-specific coverage and merge rules.
- The coverage ratio plot highlights that the COVID season is the main structural outlier; other seasons sit near parity after the API fill.

Next step: Build rolling, lagged team features to avoid lookahead bias.

Part 3: Feature Engineering

We create lagged rolling predictors (L5/L10) from team box score data to avoid lookahead bias. These features capture short-term form (L5) and medium-term form (L10) in scoring, efficiency, rebounding, turnovers, and rest.

Feature groups:

- **Scoring and defense:** points scored/allowed, expected total
- **Efficiency:** FG%, 3P%, FT%
- **Possession proxies:** rebounds, turnovers, assists
- **Scheduling context:** rest days and back-to-backs

We keep the Vegas total and spread as explicit features so we can measure how much incremental signal the box-score history adds beyond the market line.

In [80]:
```
print("=" * 70)
print("FEATURE ENGINEERING")
print("=" * 70)
features_df = create_feature_matrix(betting_data, game_data)
```

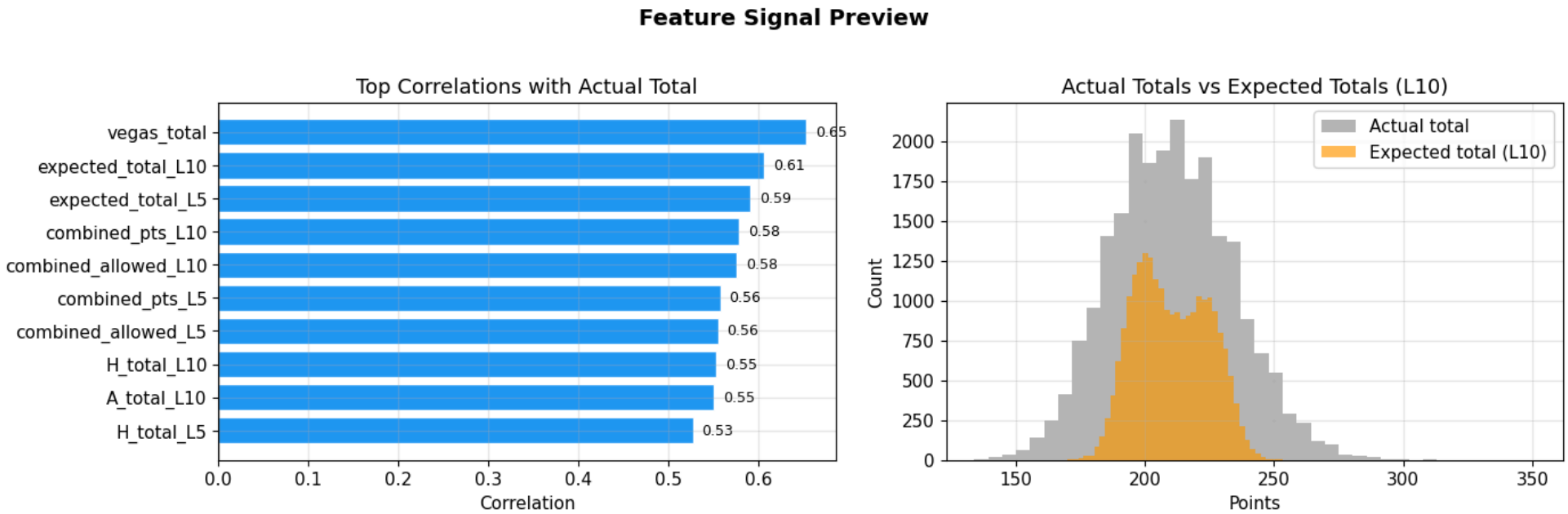
```
feature_cols = get_feature_columns(features_df)
print(f"\nTotal features: {len(feature_cols)}")
```

```
======================================================================
FEATURE ENGINEERING
======================================================================
Computing rolling features...
Feature matrix: 22,962 games (dropped 156 with insufficient history)

Total features: 80
Feature matrix: 22,962 games (dropped 156 with insufficient history)

Total features: 80
```

In [81]:

```
# Feature preview: correlations with actual total
corr = features_df[feature_cols].corrwith(features_df['actual_total']).sort_values(key=lambda s: s.abs(), ascending=False)
top_corr = corr.head(10).reset_index()
top_corr.columns = ['feature', 'corr']
print(top_corr.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

ax = axes[0]
bars = ax.barh(top_corr['feature'][::-1], top_corr['corr'][::-1],
               color=COLORS['primary'], edgecolor='white')
ax.axvline(0, color='black', lw=1)
ax.set_title('Top Correlations with Actual Total')
ax.set_xlabel('Correlation')
for bar in bars:
    ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height()/2,
            f"{bar.get_width():.2f}", va='center', fontsize=9)

ax = axes[1]
ax.hist(features_df['actual_total'], bins=40, alpha=0.75, label='Actual total', color=COLORS['neutral'])
if 'expected_total_L10' in features_df.columns:
    ax.hist(features_df['expected_total_L10'], bins=40, alpha=0.65, label='Expected total (L10)', color=COLORS['secondary'])
ax.set_title('Actual Totals vs Expected Totals (L10)')
ax.set_xlabel('Points'); ax.set_ylabel('Count')
ax.legend()

fig.suptitle('Feature Signal Preview', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('Images/feature_preview.png', dpi=150, bbox_inches='tight')
plt.show()
```

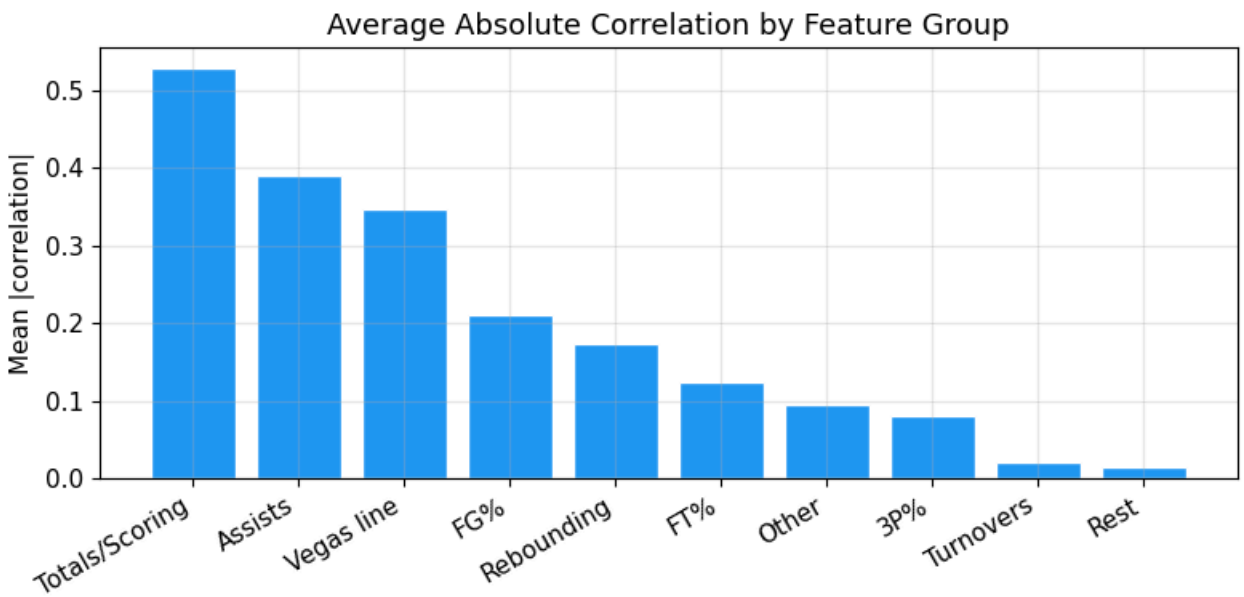
```
            feature     corr
        vegas_total 0.653305
  expected_total_L10 0.606565
   expected_total_L5 0.591672
    combined_pts_L10 0.579113
combined_allowed_L10 0.576462
     combined_pts_L5 0.557772
 combined_allowed_L5 0.555344
        H_total_L10 0.553359
        A_total_L10 0.550692
         H_total_L5 0.527349
```



In [82]:

```
# Figure: Mean absolute correlation by feature group
corr = features_df[feature_cols].corrwith(features_df['actual_total'])

def feature_group(name):
    n = name.lower()
    if 'vegas' in n:
        return 'Vegas line'
    if 'expected_total' in n or 'combined_pts' in n or 'pts_scored' in n or 'pts_allowed' in n or 'total' in n:
        return 'Totals/Scoring'
    if 'fg3' in n:
        return '3P%'
    if 'fg_pct' in n:
        return 'FG%'
    if 'ft' in n and 'pct' in n:
        return 'FT%'
    if 'reb' in n:
        return 'Rebounding'
    if 'ast' in n:
        return 'Assists'
    if 'tov' in n:
        return 'Turnovers'
    if 'rest' in n or 'days_rest' in n or 'back_to_back' in n:
        return 'Rest'
    return 'Other'

group_df = pd.DataFrame({'feature': corr.index, 'abs_corr': corr.abs().values})
group_df['group'] = group_df['feature'].apply(feature_group)
group_stats = group_df.groupby('group')['abs_corr'].mean().sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(group_stats.index, group_stats.values, color=COLORS['primary'], edgecolor='white')
ax.set_title('Average Absolute Correlation by Feature Group')
```

```
ax.set_ylabel('Mean |correlation|')
ax.set_xticklabels(group_stats.index, rotation=30, ha='right')
plt.tight_layout()
plt.savefig('Images/feature_group_corr.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- The Vegas total is still the strongest single correlate ($r \approx 0.65$), confirming market efficiency.
- Rolling expected totals and combined scoring features are next, which makes sense because they proxy pace and recent scoring environment.
- Efficiency metrics (FG%, 3P%, FT%) show smaller but consistent correlations, suggesting marginal value when combined with volume stats.
- The feature-group plot shows that totals/pace features dominate, while rest and turnover signals are smaller but nonzero.

Next step: Train and tune models on these lagged features to predict totals vs the Vegas line.

Part 4: Model Training and Evaluation

Train: Seasons 2008–2023 (~19,000 games) | **Test:** Seasons 2024–2025 (~2,600 games)

Betting strategy: If model predicts total > Vegas line, bet OVER; else UNDER. At -110 odds, need **>52.4% accuracy** to profit.

Model Set and Tuning

We start broad, compare multiple model classes, then focus on the top 2–3 for deeper analysis. Hyperparameters are tuned with time-series cross-validation (k=5 folds).

Recency weighting: We also test exponential time-decay weights so newer games matter more. The half-life is selected by cross-validated MSE on the training set. A 3-season half-life means a game three seasons old gets half the weight of a current game. We apply the same weighting scheme to every model we train so comparisons are fair.

Why 3 seasons (not 1, 2, or 4): the cross-validated error curve favors 3 seasons because it balances two forces:

- **Statistical stability:** too-short half-lives (1–2 seasons) overreact to noisy, year-specific variance.
- **Economic relevance:** too-long half-lives (4+ seasons) overweight older regimes (pace, rules, and lineup composition) that no longer match the current betting environment.

Focus Models (from this round of results)

- **PCR**: best all-bets accuracy in the test period.
- **Ridge**: strong selective-bet ROI at usable thresholds.
- **RBF + Ridge**: similar selective ROI with a nonlinear feature map.

Model Triage (theme going forward)

- **Best / focus:** PCR, Ridge, RBF+Ridge (best accuracy or best selective ROI).
- **Watch list:** Lasso, Gradient Boosting (kept through Part 5 to confirm they do not improve under thresholds).
- **Dropped after Part 4:** OLS and Elastic Net (no improvement vs Ridge, worse accuracy/ROI).

Model	Type	Role in Analysis
OLS	Linear, unregularized	Baseline — no constraints
Ridge	Linear, L2 penalty	Shrinks all coefficients — stable but keeps every feature
Lasso	Linear, L1 penalty	Sparse — zeros out irrelevant features
Elastic Net	L1 + L2 hybrid	Bridge between Ridge and Lasso
PCR (PCA + Ridge)	Linear, reduced dimension	Class method — low-rank benchmark
RBF + Ridge	Nonlinear kernel approx	Nonlinear feature expansion
Gradient Boosting	Tree ensemble	Nonlinear interactions via trees

In [55]:
```
# —— Evaluation functions ——

def train_test_split_by_season(df, test_seasons=[2024, 2025]):
    train = df[~df['season'].isin(test_seasons)].copy()
    test = df[df['season'].isin(test_seasons)].copy()
    print(f"Train: {len(train):,} games (seasons {train['season'].min()}–{train['season'].max()})")
    print(f"Test:  {len(test):,} games (seasons {test['season'].min()}–{test['season'].max()})")
    return train, test


def evaluate_predictions(y_true, y_pred, vegas_line, method_name, odds=-110):
    y_true, y_pred, vegas = map(np.array, [y_true, y_pred, vegas_line])
    rmse = np.sqrt(np.mean((y_true - y_pred) ** 2))
    correct = (y_pred > vegas) == (y_true > vegas)
    accuracy = np.mean(correct)
    lose_amt = abs(odds) / 100
    roi = (np.sum(correct) - np.sum(~correct) * lose_amt) / len(y_true) * 100
    return {
        'method': method_name,
        'rmse': rmse,
        'accuracy': accuracy,
```

```
        'roi_all_bets': roi,
        'n_total_bets': len(y_true),
    }


def sweep_confidence_thresholds(y_true_total, y_pred_total, vegas_line, thresholds=range(1, 11), odds=-110):
    y_true, y_pred, vegas = map(np.array, [y_true_total, y_pred_total, vegas_line])
    lose_amt = abs(odds) / 100
    correct = (y_pred > vegas) == (y_true > vegas)
    gap = np.abs(y_pred - vegas)
    rows = []
    for t in thresholds:
        mask = gap > t
        n = np.sum(mask)
        if n > 0:
            cc = correct[mask]
            acc = np.mean(cc)
            roi = (np.sum(cc) - np.sum(~cc) * lose_amt) / n * 100
        else:
            acc = roi = 0
        rows.append({'threshold': t, 'n_bets': n, 'accuracy': acc, 'roi': roi})
    return pd.DataFrame(rows)


def bootstrap_roi_ci(y_true_total, y_pred_total, vegas_line, threshold, odds=-110, n_boot=5000):
    y_true, y_pred, vegas = map(np.array, [y_true_total, y_pred_total, vegas_line])
    lose_amt = abs(odds) / 100
    correct = (y_pred > vegas) == (y_true > vegas)
    mask = np.abs(y_pred - vegas) > threshold
    if np.sum(mask) < 10:
        return {
            'observed_roi': 0,
            'ci_lower': 0,
            'ci_upper': 0,
            'p_value': 1.0,
            'n_bets': int(np.sum(mask)),
            'significant': False,
        }
    outcomes = np.where(correct[mask], 1.0, -lose_amt)
    np.random.seed(42)
    boot = np.array([
        np.mean(np.random.choice(outcomes, len(outcomes), replace=True)) * 100
        for _ in range(n_boot)
    ])
    return {
        'observed_roi': np.mean(outcomes) * 100,
        'ci_lower': np.percentile(boot, 2.5),
        'ci_upper': np.percentile(boot, 97.5),
        'p_value': np.mean(boot <= 0),
        'n_bets': len(outcomes),
        'significant': np.mean(boot <= 0) < 0.05,
    }


def compute_time_decay_weights(dates, half_life_days):
    age_days = (dates.max() - dates).dt.days
    return 0.5 ** (age_days / half_life_days)
```

```python
def select_half_life(X_train, y_train, dates, half_life_seasons):
    tscv = TimeSeriesSplit(n_splits=5)
    scaler = StandardScaler()
    X_s = scaler.fit_transform(X_train)
    best_hl = half_life_seasons[0]
    best_score = float('-inf')
    for hl in half_life_seasons:
        w = compute_time_decay_weights(dates, hl * 365.25).values
        scores = []
        for tr_idx, val_idx in tscv.split(X_s):
            ridge = Ridge(alpha=1.0)
            ridge.fit(X_s[tr_idx], y_train.iloc[tr_idx], sample_weight=w[tr_idx])
            pred = ridge.predict(X_s[val_idx])
            scores.append(-np.mean((y_train.iloc[val_idx] - pred) ** 2))
        score = np.mean(scores)
        if score > best_score:
            best_score = score
            best_hl = hl
    return best_hl


# —— Hyperparameter Tuning ——

def tune_hyperparameters(X_train, y_train, sample_weight=None):
    tscv = TimeSeriesSplit(n_splits=5)
    tscv_rbf = TimeSeriesSplit(n_splits=3)
    scaler = StandardScaler()
    X_s = scaler.fit_transform(X_train)
    best = {}
    fit_kwargs = {'sample_weight': sample_weight} if sample_weight is not None else {}

    print("  Tuning Ridge...")
    cv = GridSearchCV(Ridge(), {'alpha': [0.01, 0.1, 1.0, 10.0, 100.0]},
                      cv=tscv, scoring='neg_mean_squared_error')
    cv.fit(X_s, y_train, **fit_kwargs)
    best['Ridge'] = cv.best_params_

    print("  Tuning Lasso...")
    cv = GridSearchCV(Lasso(max_iter=10000), {'alpha': [0.001, 0.01, 0.1, 1.0, 10.0]},
                      cv=tscv, scoring='neg_mean_squared_error')
    cv.fit(X_s, y_train, **fit_kwargs)
    best['Lasso'] = cv.best_params_

    print("  Tuning Elastic Net...")
    cv = GridSearchCV(ElasticNet(max_iter=10000),
                      {'alpha': [0.01, 0.1, 1.0], 'l1_ratio': [0.2, 0.5, 0.8]},
                      cv=tscv, scoring='neg_mean_squared_error')
    cv.fit(X_s, y_train, **fit_kwargs)
    best['ElasticNet'] = cv.best_params_

    print("  Tuning PCR (PCA + Ridge)...")
    pcr_best, pcr_score = {'n_components': 20, 'alpha': 1.0}, float('-inf')
    for n_comp in [5, 10, 20, 40]:
        pca = PCA(n_components=n_comp, random_state=42)
        X_p = pca.fit_transform(X_s)
```

```python
        for alpha in [0.1, 1.0, 10.0]:
            scores = []
            for tr_idx, val_idx in tscv.split(X_p):
                ridge = Ridge(alpha=alpha)
                if sample_weight is not None:
                    ridge.fit(X_p[tr_idx], y_train.iloc[tr_idx], sample_weight=sample_weight[tr_idx])
                else:
                    ridge.fit(X_p[tr_idx], y_train.iloc[tr_idx])
                scores.append(-np.mean((y_train.iloc[val_idx] - ridge.predict(X_p[val_idx])) ** 2))
            score = np.mean(scores)
            if score > pcr_score:
                pcr_score = score
                pcr_best = {'n_components': n_comp, 'alpha': alpha}
    best['PCR'] = pcr_best

    print("  Tuning Gradient Boosting...")
    cv = GridSearchCV(GradientBoostingRegressor(random_state=42),
                      {'n_estimators': [50, 100, 200], 'max_depth': [2, 3, 4],
                       'learning_rate': [0.05, 0.1, 0.2]},
                      cv=tscv, scoring='neg_mean_squared_error')
    cv.fit(X_s, y_train, **fit_kwargs)
    best['GradientBoost'] = cv.best_params_

    print("  Tuning RBF + Ridge...")
    best_score, best_rbf = float('-inf'), {'gamma': 0.001, 'n_components': 500, 'alpha': 1.0}
    gamma_grid = [1e-5, 3e-5, 1e-4, 3e-4, 1e-3, 3e-3, 1e-2]
    n_comp_grid = [200, 500, 1000, 1500]
    alpha_grid = [0.01, 0.1, 1.0, 10.0]
    for gamma in gamma_grid:
        for n_comp in n_comp_grid:
            for alpha in alpha_grid:
                scores = []
                for tr_idx, val_idx in tscv_rbf.split(X_s):
                    rbf = RBFSampler(gamma=gamma, n_components=n_comp, random_state=42)
                    X_tr_r = rbf.fit_transform(X_s[tr_idx])
                    X_va_r = rbf.transform(X_s[val_idx])
                    ridge = Ridge(alpha=alpha)
                    if sample_weight is not None:
                        ridge.fit(X_tr_r, y_train.iloc[tr_idx], sample_weight=sample_weight[tr_idx])
                    else:
                        ridge.fit(X_tr_r, y_train.iloc[tr_idx])
                    scores.append(-np.mean((y_train.iloc[val_idx] - ridge.predict(X_va_r)) ** 2))
                if np.mean(scores) > best_score:
                    best_score = np.mean(scores)
                    best_rbf = {'gamma': gamma, 'n_components': n_comp, 'alpha': alpha}
    best['RBF+Ridge'] = best_rbf

    print("\nBest hyperparameters:")
    for m, p in best.items():
        print(f"  {m}: {p}")
    return best


# —— Train all models with tuned params ——

def train_all_models(X_train, y_train, X_test, y_test, vegas_test, best_params, sample_weight=None):
    scaler = StandardScaler()
```

```python
    X_tr = scaler.fit_transform(X_train)
    X_te = scaler.transform(X_test)
    results, predictions = [], {}
    fit_kwargs = {'sample_weight': sample_weight} if sample_weight is not None else {}

    configs = [
        ('OLS', LinearRegression()),
        ('Ridge', Ridge(**best_params['Ridge'])),
        ('Lasso', Lasso(**best_params['Lasso'], max_iter=10000)),
        ('Elastic Net', ElasticNet(**best_params['ElasticNet'], max_iter=10000)),
        ('Gradient Boosting', GradientBoostingRegressor(**best_params['GradientBoost'], random_state=42)),
    ]
    for name, model in configs:
        model.fit(X_tr, y_train, **fit_kwargs)
        pred = model.predict(X_te)
        results.append(evaluate_predictions(y_test, pred, vegas_test, name))
        predictions[name] = pred
        print(f"  {name}: RMSE={results[-1]['rmse']:.2f}, Acc={results[-1]['accuracy']:.1%}, ROI={results[-1]['roi_all_bets']:.2f}%")

    # PCR (PCA + Ridge)
    pcr_params = best_params['PCR']
    pca = PCA(n_components=pcr_params['n_components'], random_state=42)
    X_tr_p = pca.fit_transform(X_tr)
    X_te_p = pca.transform(X_te)
    pcr = Ridge(alpha=pcr_params['alpha'])
    if sample_weight is not None:
        pcr.fit(X_tr_p, y_train, sample_weight=sample_weight)
    else:
        pcr.fit(X_tr_p, y_train)
    pred_pcr = pcr.predict(X_te_p)
    results.append(evaluate_predictions(y_test, pred_pcr, vegas_test, 'PCR'))
    predictions['PCR'] = pred_pcr
    print(f"  PCR: RMSE={results[-1]['rmse']:.2f}, Acc={results[-1]['accuracy']:.1%}, ROI={results[-1]['roi_all_bets']:.2f}%")

    # RBF + Ridge
    rp = best_params['RBF+Ridge']
    rbf = RBFSampler(gamma=rp['gamma'], n_components=rp['n_components'], random_state=42)
    X_tr_rbf = rbf.fit_transform(X_tr)
    X_te_rbf = rbf.transform(X_te)
    ridge_rbf = Ridge(alpha=rp['alpha'])
    if sample_weight is not None:
        ridge_rbf.fit(X_tr_rbf, y_train, sample_weight=sample_weight)
    else:
        ridge_rbf.fit(X_tr_rbf, y_train)
    pred_rbf = ridge_rbf.predict(X_te_rbf)
    results.append(evaluate_predictions(y_test, pred_rbf, vegas_test, 'RBF+Ridge'))
    predictions['RBF+Ridge'] = pred_rbf
    print(f"  RBF+Ridge: RMSE={results[-1]['rmse']:.2f}, Acc={results[-1]['accuracy']:.1%}, ROI={results[-1]['roi_all_bets']:.2f}%")

    # Vegas benchmark
    vegas_rmse = np.sqrt(np.mean((np.array(y_test) - np.array(vegas_test)) ** 2))
    over_rate = np.mean(np.array(y_test) > np.array(vegas_test))
    results.append({
        'method': 'Vegas (benchmark)',
        'rmse': vegas_rmse,
        'accuracy': max(over_rate, 1 - over_rate),
        'roi_all_bets': 0,
```

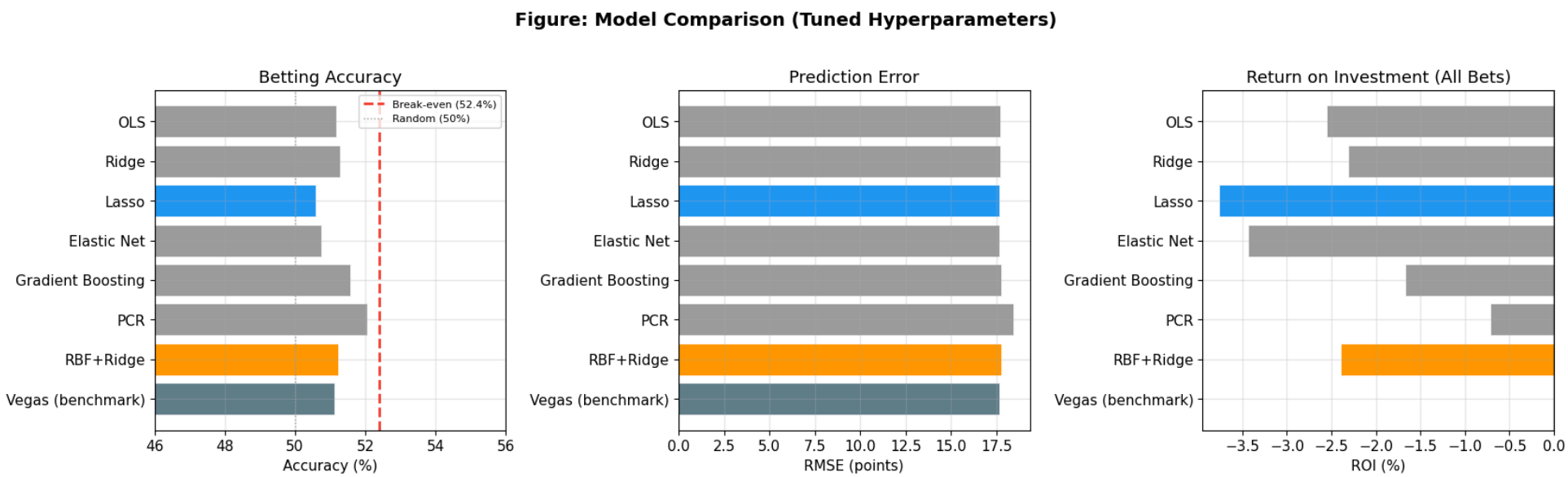
```python
        'n_total_bets': len(y_test),
    })
    predictions['Vegas'] = np.array(vegas_test)
    return pd.DataFrame(results), predictions


# ── Execute ──
print("=" * 70)
print("MODEL TRAINING")
print("=" * 70)

train_df, test_df = train_test_split_by_season(features_df)
X_train = train_df[feature_cols]
X_test  = test_df[feature_cols]
y_train = train_df['actual_total']
y_test  = test_df['actual_total']
vegas_test = test_df['vegas_total']
print(f"Features: {len(feature_cols)}\n")

USE_TIME_DECAY = True
HALF_LIFE_SEASONS = [0.5, 1, 2, 3]
if USE_TIME_DECAY:
    best_half_life = select_half_life(X_train, y_train, train_df['date'], HALF_LIFE_SEASONS)
    half_life_days = best_half_life * 365.25
    train_weights = compute_time_decay_weights(train_df['date'], half_life_days).values
    print(f"Time-decay weights: half-life = {best_half_life} seasons")
else:
    train_weights = None

print("Hyperparameter Tuning (5-fold Time-Series CV)...")
best_params = tune_hyperparameters(X_train, y_train, sample_weight=train_weights)

print("\nTraining with Tuned Parameters...")
results_tuned, preds_tuned = train_all_models(
    X_train, y_train, X_test, y_test, vegas_test, best_params, sample_weight=train_weights
 )

print("\n" + "=" * 70)
print("MODEL RESULTS")
print("=" * 70)
print(results_tuned.to_string(index=False))
```

```
======================================================================
MODEL TRAINING
======================================================================
Train: 20,341 games (seasons 2008-2023)
Test:  2,621 games (seasons 2024-2025)
Features: 80

Time-decay weights: half-life = 3 seasons
Hyperparameter Tuning (5-fold Time-Series CV)...
  Tuning Ridge...
  Tuning Lasso...
  Tuning Elastic Net...
  Tuning PCR (PCA + Ridge)...
  Tuning Gradient Boosting...
  Tuning RBF + Ridge...

Best hyperparameters:
  Ridge: {'alpha': 1.0}
  Lasso: {'alpha': 0.1}
  ElasticNet: {'alpha': 0.1, 'l1_ratio': 0.8}
  PCR: {'n_components': 40, 'alpha': 10.0}
  GradientBoost: {'learning_rate': 0.05, 'max_depth': 2, 'n_estimators': 100}
  RBF+Ridge: {'gamma': 0.0003, 'n_components': 1000, 'alpha': 0.01}

Training with Tuned Parameters...
  OLS: RMSE=17.70, Acc=51.2%, ROI=-2.56%
  Ridge: RMSE=17.70, Acc=51.3%, ROI=-2.32%
  Lasso: RMSE=17.67, Acc=50.6%, ROI=-3.76%
  Elastic Net: RMSE=17.68, Acc=50.7%, ROI=-3.44%
  Gradient Boosting: RMSE=17.77, Acc=51.6%, ROI=-1.67%
  PCR: RMSE=18.41, Acc=52.0%, ROI=-0.71%
  RBF+Ridge: RMSE=17.75, Acc=51.2%, ROI=-2.40%

======================================================================
MODEL RESULTS
======================================================================
           method      rmse  accuracy  roi_all_bets  n_total_bets
              OLS 17.699312  0.511637     -2.556276          2621
            Ridge 17.699092  0.512781     -2.315910          2621
            Lasso 17.669550  0.505914     -3.758108          2621
      Elastic Net 17.679007  0.507440     -3.437619          2621
Gradient Boosting 17.770254  0.515834     -1.674933          2621
              PCR 18.412888  0.520412     -0.713468          2621
        RBF+Ridge 17.754007  0.512400     -2.396032          2621
Vegas (benchmark) 17.649625  0.511255      0.000000          2621
```

In [83]:

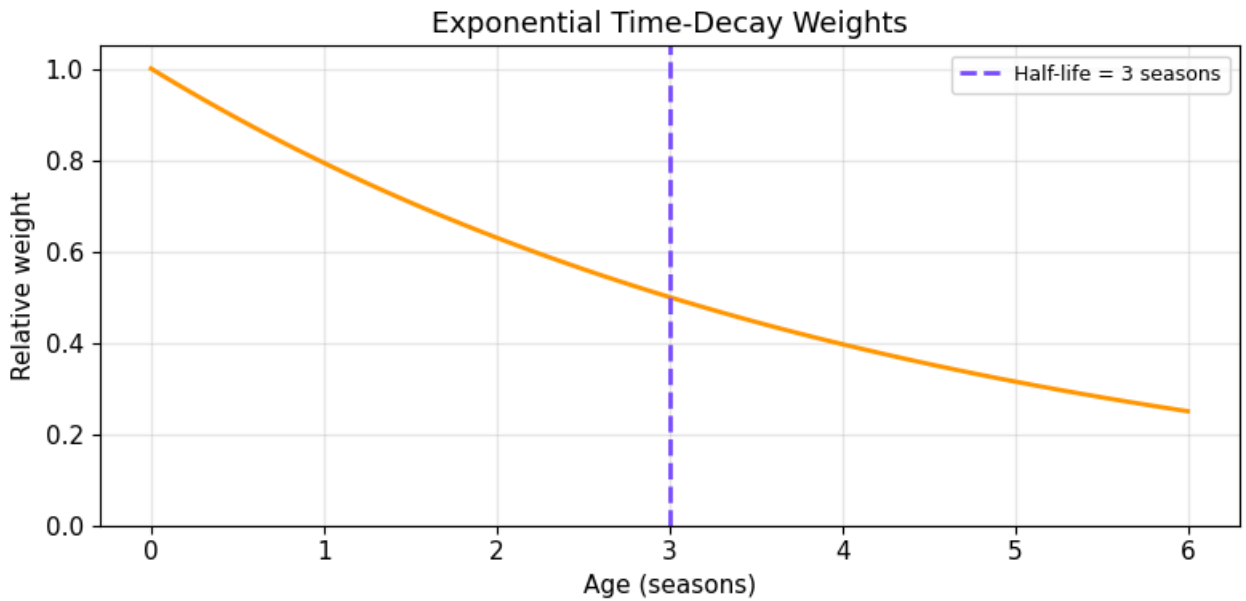
```python
# Figure: Model comparison (Part 4)
fig, axes = plt.subplots(1, 3, figsize=(17, 5))
methods = results_tuned['method'].tolist()
x = np.arange(len(methods))

method_colors = []
for m in methods:
    if m == 'Lasso':        method_colors.append(COLORS['primary'])
    elif m == 'RBF+Ridge':  method_colors.append(COLORS['secondary'])
    elif 'Vegas' in m:      method_colors.append(COLORS['vegas'])
    else:                   method_colors.append(COLORS['neutral'])
```

```
ax = axes[0]
acc = results_tuned['accuracy'] * 100
ax.barh(x, acc, color=method_colors, edgecolor='white', linewidth=0.5)
ax.axvline(52.4, color=COLORS['breakeven'], ls='--', lw=2, label='Break-even (52.4%)')
ax.axvline(50, color=COLORS['neutral'], ls=':', lw=1, label='Random (50%)')
ax.set_yticks(x); ax.set_yticklabels(methods)
ax.set_xlabel('Accuracy (%)'); ax.set_title('Betting Accuracy')
ax.set_xlim(46, 56); ax.legend(fontsize=8); ax.invert_yaxis()

ax = axes[1]
ax.barh(x, results_tuned['rmse'], color=method_colors, edgecolor='white', linewidth=0.5)
ax.set_yticks(x); ax.set_yticklabels(methods)
ax.set_xlabel('RMSE (points)'); ax.set_title('Prediction Error')
ax.invert_yaxis()

ax = axes[2]
roi_vals = results_tuned['roi_all_bets']
ax.barh(x, roi_vals, color=method_colors, edgecolor='white', linewidth=0.5)
ax.axvline(0, color='black', lw=1)
ax.set_yticks(x); ax.set_yticklabels(methods)
ax.set_xlabel('ROI (%)'); ax.set_title('Return on Investment (All Bets)')
ax.invert_yaxis()

fig.suptitle('Figure: Model Comparison (Tuned Hyperparameters)', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('Images/model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [84]:
```
# Figure: Time-decay weighting curve
ages = np.linspace(0, 6, 61)
weights = 0.5 ** (ages / best_half_life)

fig, ax = plt.subplots(figsize=(8, 4))
```

```
ax.plot(ages, weights, color=COLORS['secondary'], lw=2)
ax.axvline(best_half_life, color=COLORS['highlight'], ls='--', lw=2, label=f'Half-life = {best_half_life} seasons')
ax.set_title('Exponential Time-Decay Weights')
ax.set_xlabel('Age (seasons)'); ax.set_ylabel('Relative weight')
ax.set_ylim(0, 1.05)
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig('Images/time_decay_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- Time-decay weighting selects a 3-season half-life, meaning recent seasons get more emphasis in training (see weight curve).
- PCR is the best all-bets accuracy here (~52.0%), but still below break-even after the vig.
- Ridge and RBF+Ridge emerge as the best candidates for selective betting, so we focus on them next.
- OLS and Elastic Net are dropped after this section because they are dominated by Ridge on accuracy and ROI.

Why some models outperform:

- **Linear shrinkage models (Ridge/PCR)** tend to win in efficient markets because they reduce noise and avoid overfitting.
- **Nonlinear models** can help when interactions exist, but they often overfit small edges and are unstable out of sample.

Next step: Explore selective betting to see if confidence thresholds improve ROI.

> **Model Output Summary (after rerun)**

- Best all-bets accuracy is PCR at 52.0% (still below break-even).
- Time-decay weighting selects a 3-season half-life and is applied to all models during training.
- Ridge and RBF+Ridge show the strongest selective-bet ROI pockets, so they become the focus set.

- OLS and Elastic Net are dropped; Lasso and Gradient Boosting stay on a watch list through Part 5 only.

Part 5: Confidence Threshold Analysis

Betting every game is a losing strategy — the vig (-110 odds) requires >52.4% accuracy to break even. We adopt **selective betting:** only wager when the model strongly disagrees with Vegas.

The **confidence threshold** is the minimum gap (in points) between prediction and Vegas line. Higher threshold = fewer but more confident bets.

We first show a single-model threshold sweep (RBF+Ridge) to explain the mechanics. Then we compare ROI and bet volume across several models, while keeping our interpretation centered on the current top performers (PCR, Ridge, RBF+Ridge). Lasso and Gradient Boosting are included only to confirm whether they can compete under thresholds.

Important: ROIs at very high thresholds can be unstable when the number of bets is tiny. We still report thresholds 1–10 for transparency, but only treat thresholds with a meaningful sample (>= 30 bets) as viable.

In [85]:
```
print("=" * 70)
print("CONFIDENCE THRESHOLD SWEEP")
print("=" * 70)

MIN_BETS = 30
threshold_results = {}
for name in ['RBF+Ridge', 'Lasso', 'Ridge', 'PCR', 'Gradient Boosting']:
    if name in preds_tuned:
        threshold_results[name] = sweep_confidence_thresholds(
            y_test, preds_tuned[name], vegas_test, thresholds=range(1, 11))

for name, tdf in threshold_results.items():
    print(f"\n{name}:")
    tdf_disp = tdf.copy()
    tdf_disp['note'] = np.where(tdf_disp['n_bets'] < 10, 'low_n', '')
    print(tdf_disp.to_string(index=False))

# Best threshold per model (highest ROI with >= MIN_BETS)
print(f"\n── Optimal Thresholds (min {MIN_BETS} bets) ──")
best_thresholds = {}
for name, tdf in threshold_results.items():
    viable = tdf[tdf['n_bets'] >= MIN_BETS]
    if len(viable) > 0:
        best_row = viable.loc[viable['roi'].idxmax()]
        best_thresholds[name] = int(best_row['threshold'])
        print(f"  {name}: threshold={best_row['threshold']:.0f}, "
              f"ROI={best_row['roi']:.2f}%, acc={best_row['accuracy']:.1%}, n={best_row['n_bets']:.0f} bets")
```

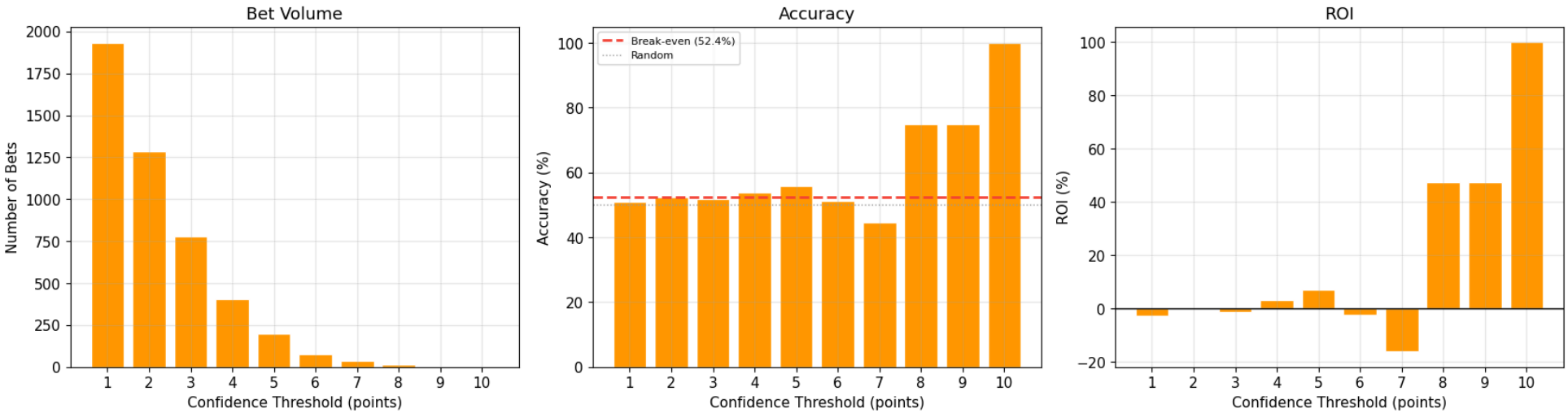
```
======================================================================
CONFIDENCE THRESHOLD SWEEP
======================================================================

RBF+Ridge:
 threshold  n_bets  accuracy        roi  note
         1    1932  0.510870  -2.717391
         2    1286  0.523328  -0.101089
         3     779  0.517330  -1.360719
         4     408  0.539216   3.235294
         5     197  0.558376   7.258883
         6      78  0.512821  -2.307692
         7      38  0.447368 -16.052632
         8      16  0.750000  47.500000
         9       4  0.750000  47.500000 low_n
        10       2  1.000000 100.000000 low_n

Lasso:
 threshold  n_bets  accuracy        roi  note
         1    1276  0.519592  -0.885580
         2     436  0.513761  -2.110092
         3     104  0.519231  -0.961538
         4      17  0.470588 -11.176471
         5       1  1.000000 100.000000 low_n
         6       0  0.000000   0.000000 low_n
         7       0  0.000000   0.000000 low_n
         8       0  0.000000   0.000000 low_n
         9       0  0.000000   0.000000 low_n
        10       0  0.000000   0.000000 low_n

Ridge:
 threshold  n_bets  accuracy        roi  note
         1    1691  0.516263  -1.584861
         2     926  0.528078   0.896328
         3     409  0.523227  -0.122249
         4     151  0.576159  10.993377
         5      37  0.567568   9.189189
         6      11  0.545455   4.545455
         7       3  0.333333 -40.000000 low_n
         8       0  0.000000   0.000000 low_n
         9       0  0.000000   0.000000 low_n
        10       0  0.000000   0.000000 low_n

PCR:
 threshold  n_bets  accuracy        roi note
         1    2259  0.519256  -0.956175
         2    1907  0.522286  -0.319874
         3    1575  0.513651  -2.133333
         4    1251  0.515588  -1.726619
         5     972  0.519547  -0.895062
         6     738  0.514905  -1.869919
         7     542  0.522140  -0.350554
         8     393  0.496183  -5.801527
         9     287  0.473868 -10.487805
        10     197  0.426396 -20.456853

Gradient Boosting:
```

```
 threshold  n_bets  accuracy        roi note
         1    1319  0.517817  -1.258529
         2     594  0.500000  -5.000000
         3     266  0.496241  -5.789474
         4     132  0.507576  -3.409091
         5      73  0.465753 -12.191781
         6      47  0.446809 -16.170213
         7      33  0.393939 -27.272727
         8      23  0.478261  -9.565217
         9      19  0.526316   0.526316
        10      17  0.588235  13.529412

── Optimal Thresholds (min 30 bets) ──
  RBF+Ridge: threshold=5, ROI=7.26%, acc=55.8%, n=197 bets
  Lasso: threshold=1, ROI=-0.89%, acc=52.0%, n=1276 bets
  Ridge: threshold=4, ROI=10.99%, acc=57.6%, n=151 bets
  PCR: threshold=2, ROI=-0.32%, acc=52.2%, n=1907 bets
  Gradient Boosting: threshold=1, ROI=-1.26%, acc=51.8%, n=1319 bets
```

In [86]:

```python
# Figure: Threshold sweep (single model)
fig, axes = plt.subplots(1, 3, figsize=(17, 5))
sweep_model = 'RBF+Ridge'
tdf = threshold_results[sweep_model]

ax = axes[0]
ax.bar(tdf['threshold'], tdf['n_bets'], color=COLORS['secondary'], edgecolor='white')
ax.set_xlabel('Confidence Threshold (points)'); ax.set_ylabel('Number of Bets')
ax.set_title('Bet Volume'); ax.set_xticks(tdf['threshold'])

ax = axes[1]
ax.bar(tdf['threshold'], tdf['accuracy']*100, color=COLORS['secondary'], edgecolor='white')
ax.axhline(52.4, color=COLORS['breakeven'], ls='--', lw=2, label='Break-even (52.4%)')
ax.axhline(50, color=COLORS['neutral'], ls=':', lw=1, label='Random')
ax.set_xlabel('Confidence Threshold (points)'); ax.set_ylabel('Accuracy (%)')
ax.set_title('Accuracy'); ax.set_xticks(tdf['threshold']); ax.legend(fontsize=8)

ax = axes[2]
ax.bar(tdf['threshold'], tdf['roi'], color=COLORS['secondary'], edgecolor='white')
ax.axhline(0, color='black', lw=1)
ax.set_xlabel('Confidence Threshold (points)'); ax.set_ylabel('ROI (%)')
ax.set_title('ROI'); ax.set_xticks(tdf['threshold'])

fig.suptitle(f'Figure: Confidence Threshold Analysis ({sweep_model})', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('Images/threshold_sweep.png', dpi=150, bbox_inches='tight')
plt.show()
```

Figure: Confidence Threshold Analysis (RBF+Ridge)



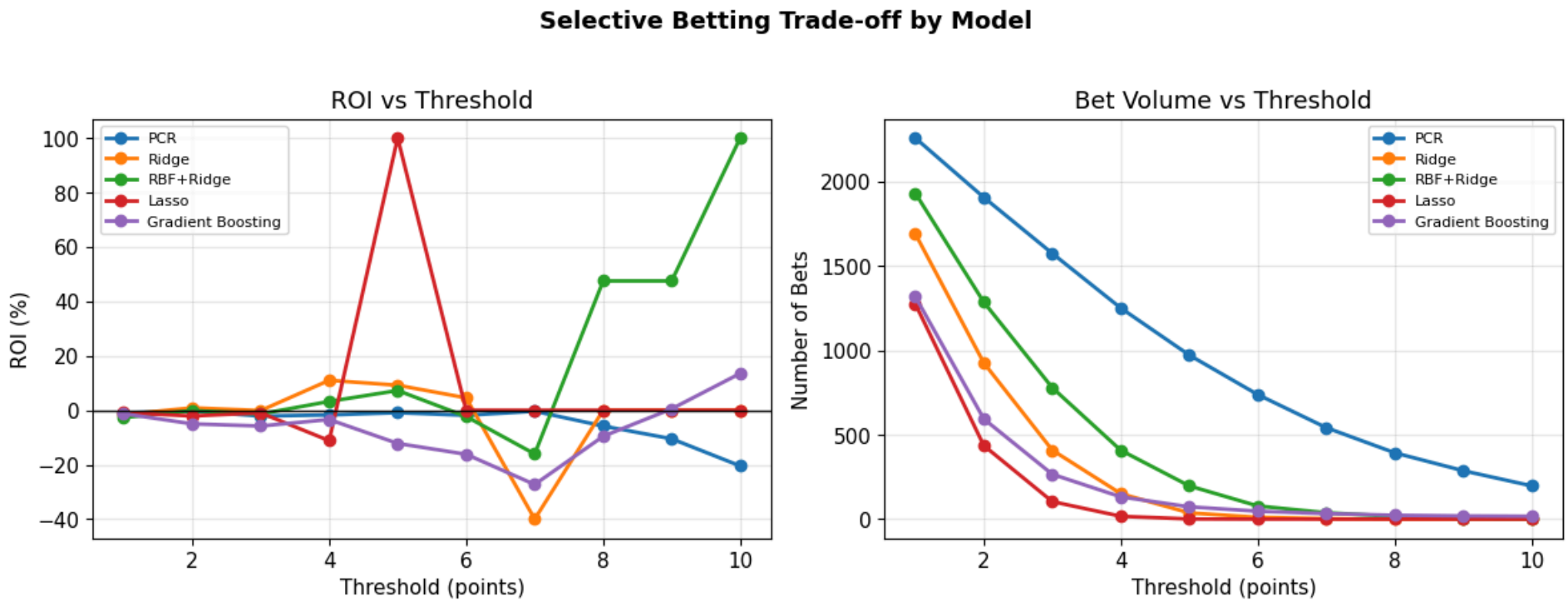
Observations:

- Higher thresholds sharply reduce bet volume, which explains the instability at very large thresholds.
- RBF+Ridge and Ridge show the best mid-threshold ROI (t=5 and t=4), but samples are still modest.
- This single-model figure is only to show how thresholds work; model-to-model comparisons follow next.

Next step: Compare ROI vs bet volume across models to see the trade-off.

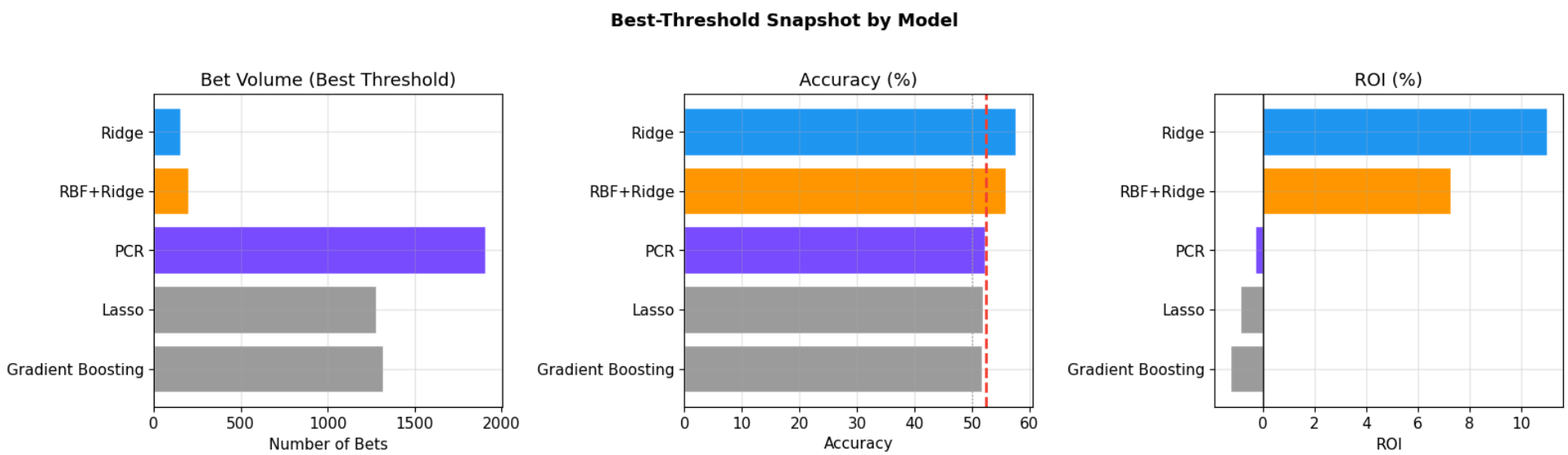
In [87]:
```
# Threshold comparison across models
plot_models = ['PCR', 'Ridge', 'RBF+Ridge', 'Lasso', 'Gradient Boosting']
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

ax = axes[0]
for m in plot_models:
    if m in threshold_results:
        ax.plot(threshold_results[m]['threshold'], threshold_results[m]['roi'],
                marker='o', lw=2, label=m)
ax.axhline(0, color='black', lw=1)
ax.set_title('ROI vs Threshold')
ax.set_xlabel('Threshold (points)'); ax.set_ylabel('ROI (%)')
ax.legend(fontsize=8)

ax = axes[1]
for m in plot_models:
    if m in threshold_results:
        ax.plot(threshold_results[m]['threshold'], threshold_results[m]['n_bets'],
                marker='o', lw=2, label=m)
ax.set_title('Bet Volume vs Threshold')
ax.set_xlabel('Threshold (points)'); ax.set_ylabel('Number of Bets')
ax.legend(fontsize=8)

fig.suptitle('Selective Betting Trade-off by Model', fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
```

```
plt.savefig('Images/threshold_tradeoff.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [88]:
```
# Figure: Best threshold summary across models
focus_models = ['PCR', 'Ridge', 'RBF+Ridge']
summary_rows = []
for name, tdf in threshold_results.items():
    if name in best_thresholds:
        t = best_thresholds[name]
        row = tdf[tdf['threshold'] == t].iloc[0]
        summary_rows.append({
            'model': name,
            'threshold': int(t),
            'n_bets': int(row['n_bets']),
            'accuracy': row['accuracy'] * 100,
            'roi': row['roi'],
        })
summary_df = pd.DataFrame(summary_rows)

def model_color(name):
    if name == 'RBF+Ridge':
        return COLORS['secondary']
    if name == 'Ridge':
        return COLORS['primary']
    if name == 'PCR':
        return COLORS['highlight']
    return COLORS['neutral']

if not summary_df.empty:
```

```python
summary_df = summary_df.sort_values('roi', ascending=False).reset_index(drop=True)
colors = [model_color(m) for m in summary_df['model']]
y = np.arange(len(summary_df))

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

ax = axes[0]
ax.barh(y, summary_df['n_bets'], color=colors, edgecolor='white')
ax.set_yticks(y); ax.set_yticklabels(summary_df['model'])
ax.set_title('Bet Volume (Best Threshold)')
ax.set_xlabel('Number of Bets')
ax.invert_yaxis()

ax = axes[1]
ax.barh(y, summary_df['accuracy'], color=colors, edgecolor='white')
ax.axvline(52.4, color=COLORS['breakeven'], ls='--', lw=2)
ax.axvline(50, color=COLORS['neutral'], ls=':', lw=1)
ax.set_yticks(y); ax.set_yticklabels(summary_df['model'])
ax.set_title('Accuracy (%)')
ax.set_xlabel('Accuracy')
ax.invert_yaxis()

ax = axes[2]
ax.barh(y, summary_df['roi'], color=colors, edgecolor='white')
ax.axvline(0, color='black', lw=1)
ax.set_yticks(y); ax.set_yticklabels(summary_df['model'])
ax.set_title('ROI (%)')
ax.set_xlabel('ROI')
ax.invert_yaxis()

fig.suptitle('Best-Threshold Snapshot by Model', fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('Images/best_threshold_snapshot.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- The model trade-off plot shows ROI peaks at mid thresholds, but volume drops fast across all models.
- The best-threshold snapshot makes the ranking explicit: Ridge and RBF+Ridge lead ROI; PCR leads accuracy.
- Lasso and Gradient Boosting do not produce stable, positive ROI at usable volumes and are dropped after this section.

Next step: Test whether the focus models' selective-bet ROIs are statistically significant.

In [89]:
```
# Diagnostics: prediction gap distribution for selected model
diag_model = 'RBF+Ridge'
if diag_model in preds_tuned:
    gap = np.abs(np.array(preds_tuned[diag_model]) - np.array(vegas_test))
    qs = np.percentile(gap, [50, 75, 90, 95, 97.5, 99])
    print(f"\nGap diagnostics ({diag_model}):")
    print(f"  max |pred - vegas|: {gap.max():.2f}")
    print("  percentiles:")
    for p, q in zip([50, 75, 90, 95, 97.5, 99], qs):
        print(f"    p{p:>4}: {q:.2f}")
    print("  counts by threshold:")
    for t in range(1, 11):
        print(f"    > {t}: {(gap > t).sum()}")
else:
    print("Gap diagnostics skipped (model not found).")
```

```
Gap diagnostics (RBF+Ridge):
  max |pred - vegas|: 10.92
  percentiles:
    p  50: 1.95
    p  75: 3.29
    p  90: 4.64
    p  95: 5.36
    p97.5: 6.23
    p  99: 7.31
  counts by threshold:
    > 1: 1932
    > 2: 1286
    > 3: 779
    > 4: 408
    > 5: 197
    > 6: 78
    > 7: 38
    > 8: 16
    > 9: 4
    > 10: 2
```

Part 6: Statistical Significance

Is the profit real or just luck? We use **bootstrap resampling** (5,000 iterations) to build a 95% confidence interval for ROI and test if it's significantly above zero.

From here onward we carry forward only the focus set (PCR, Ridge, RBF+Ridge). Lasso and Gradient Boosting are dropped after Part 5 because their selective ROI is weak and unstable.

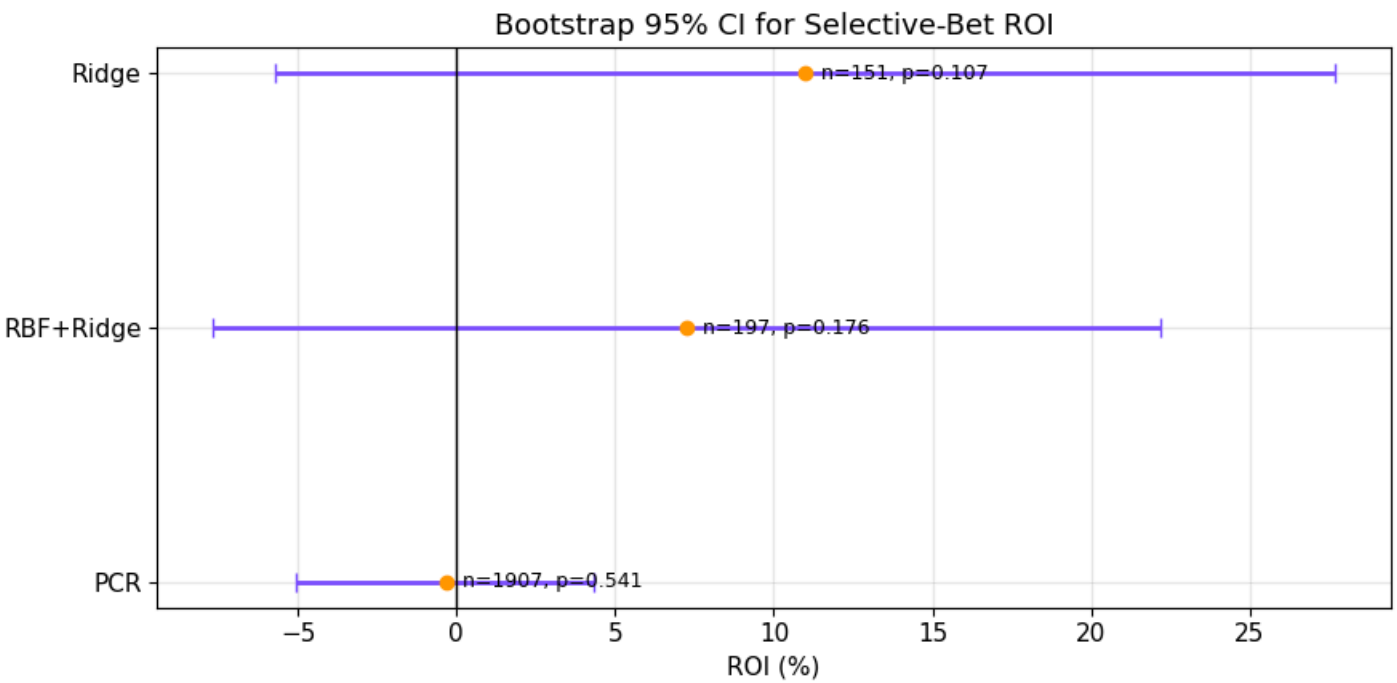
We test at the **optimal threshold** identified above — the one that maximizes ROI with a meaningful sample. The CI plot shows uncertainty around those selective-bet ROIs.

In [90]:
```
print("=" * 70)
print("STATISTICAL SIGNIFICANCE (Bootstrap, 5000 iterations)")
print("=" * 70)

focus_models = ['PCR', 'Ridge', 'RBF+Ridge']
boot_results = {}
for name in focus_models:
    if name in best_thresholds and name in preds_tuned:
        t = best_thresholds[name]
        b = bootstrap_roi_ci(y_test, preds_tuned[name], vegas_test, threshold=t)
        boot_results[name] = b
        sig = "YES" if b['significant'] else "NO"
        print(f"\n  {name} (threshold={t}):")
        print(f"    Observed ROI:  {b['observed_roi']:+.2f}%")
        print(f"    95% CI:        [{b['ci_lower']:+.2f}%, {b['ci_upper']:+.2f}%]")
        print(f"    P-value:       {b['p_value']:.4f}")
        print(f"    N bets:        {b['n_bets']}")
        print(f"    Significant:   {sig}")
    else:
        print(f"\n  {name}: skipped (no viable threshold).")
```

```
======================================================================
STATISTICAL SIGNIFICANCE (Bootstrap, 5000 iterations)
======================================================================

  PCR (threshold=2):
    Observed ROI:  -0.32%
    95% CI:        [-5.06%, +4.31%]
    P-value:       0.5410
    N bets:        1907
    Significant:   NO

  Ridge (threshold=4):
    Observed ROI:  +10.99%
    95% CI:        [-5.70%, +27.68%]
    P-value:       0.1072
    N bets:        151
    Significant:   NO

  RBF+Ridge (threshold=5):
    Observed ROI:  +7.26%
    95% CI:        [-7.66%, +22.18%]
    P-value:       0.1764
    N bets:        197
    Significant:   NO
```

In [91]:
```
# Plot bootstrap confidence intervals
if boot_results:
    ci_rows = []
    for m, b in boot_results.items():
        ci_rows.append({
            'model': m,
            'roi': b['observed_roi'],
            'ci_lower': b['ci_lower'],
            'ci_upper': b['ci_upper'],
```

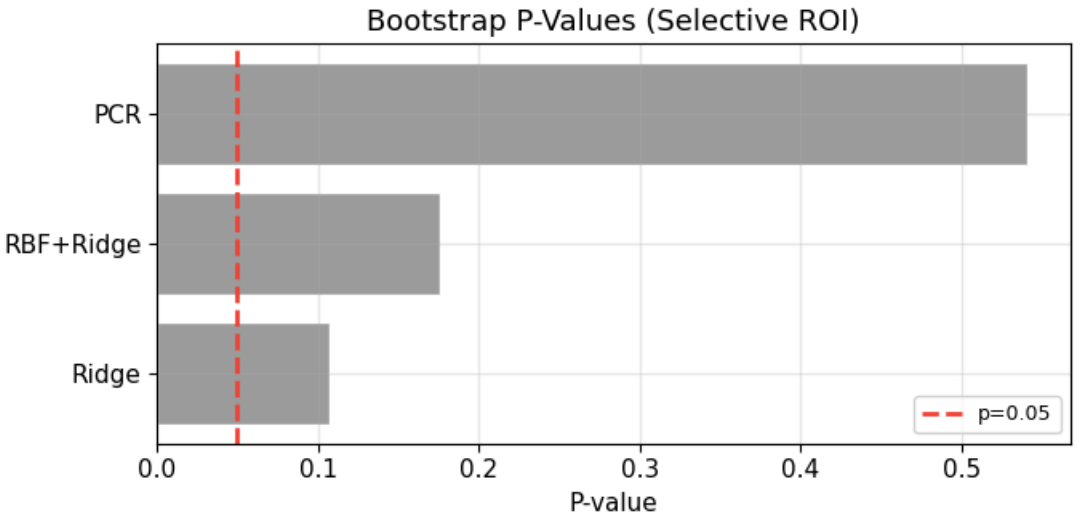
```
            'n_bets': b['n_bets'],
            'p_value': b['p_value'],
        })
    ci_df = pd.DataFrame(ci_rows).sort_values('roi')

    fig, ax = plt.subplots(figsize=(9, 4.5))
    ax.errorbar(ci_df['roi'], ci_df['model'],
xerr=[ci_df['roi'] - ci_df['ci_lower'], ci_df['ci_upper'] - ci_df['roi']],
fmt='o', color=COLORS['secondary'], ecolor=COLORS['highlight'], elinewidth=2, capsize=4)
    ax.axvline(0, color='black', lw=1)
    ax.set_xlabel('ROI (%)')
    ax.set_title('Bootstrap 95% CI for Selective-Bet ROI')
    for row in ci_df.itertuples():
        ax.text(row.roi + 0.5, row.model, f"n={row.n_bets}, p={row.p_value:.3f}", va='center', fontsize=9)
    plt.tight_layout()
    plt.savefig('Images/bootstrap_ci_models.png', dpi=150, bbox_inches='tight')
    plt.show()
```



In [92]:

```
# Figure: P-values for focus models
if boot_results:
    pv_rows = []
    for m, b in boot_results.items():
        pv_rows.append({'model': m, 'p_value': b['p_value']})
    pv_df = pd.DataFrame(pv_rows).sort_values('p_value')

    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.barh(pv_df['model'], pv_df['p_value'], color=COLORS['neutral'], edgecolor='white')
    ax.axvline(0.05, color=COLORS['loss'], ls='--', lw=2, label='p=0.05')
    ax.set_title('Bootstrap P-Values (Selective ROI)')
    ax.set_xlabel('P-value')
```

```
    ax.legend(fontsize=9)
    plt.tight_layout()
    plt.savefig('Images/bootstrap_pvalues.png', dpi=150, bbox_inches='tight')
    plt.show()
```



Observations:

- All confidence intervals include zero, so none of the selective strategies are statistically significant.
- Ridge and RBF+Ridge have the highest point estimates, but uncertainty remains wide.
- PCR remains the accuracy benchmark, but it does not deliver reliable selective ROI.
- The p-value bars reinforce that no focus model clears conventional significance thresholds.

Next step: Summarize what these results imply and what we could change to try to reach profitability.

Part 7: Recap and Path to Profit

Recap (Parts 1–6):

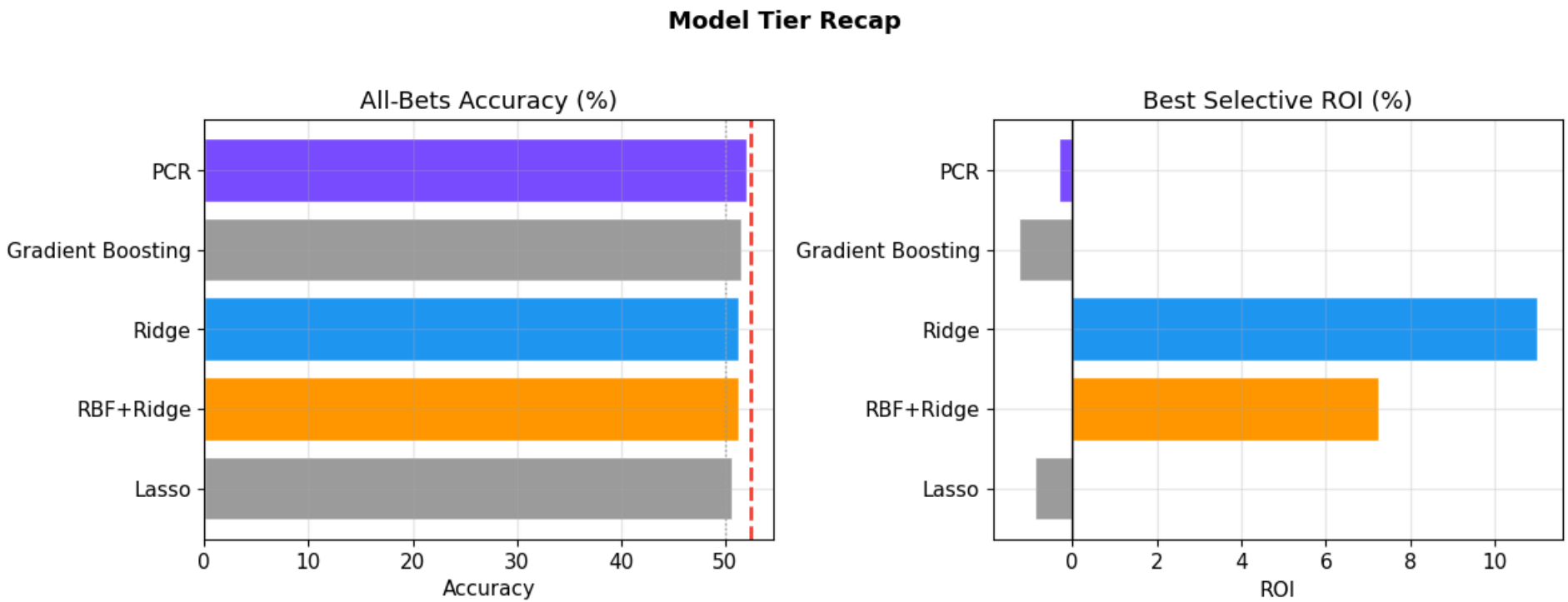
- Data coverage is strong after the API fill, with 2020 showing a COVID-driven dip.
- Features add signal beyond Vegas, but the market line remains the dominant predictor.
- No model beats break-even on all bets; selective betting creates positive ROI pockets but lacks statistical significance.
- Focus set: PCR (best accuracy) plus Ridge and RBF+Ridge (best selective ROI). Dropped: OLS, Elastic Net, Lasso, Gradient Boosting.

What might create a real edge next:

- **Better inputs:** injuries, travel, rest quality, pace, and lineup continuity can shift totals quickly.
- **Market timing:** incorporate line movement and bet at open vs close to capture stale numbers.
- **Target shift:** predict total minus Vegas line (edge) instead of raw total to reduce noise.
- **Calibration:** map prediction gaps to calibrated win probabilities before sizing bets.

- **Regime awareness:** allow different thresholds or models for early season vs late season and playoffs.

In [93]:
```
# Figure: Model tier recap (accuracy vs selective ROI)
summary_rows = []
for name in ['PCR', 'Ridge', 'RBF+Ridge', 'Lasso', 'Gradient Boosting']:
    if name in results_tuned['method'].values and name in threshold_results:
        acc = results_tuned[results_tuned['method'] == name]['accuracy'].values[0] * 100
        t = best_thresholds.get(name)
        roi = np.nan
        if t is not None:
            row = threshold_results[name][threshold_results[name]['threshold'] == t].iloc[0]
            roi = row['roi']
        summary_rows.append({'model': name, 'accuracy': acc, 'roi': roi})
summary_df = pd.DataFrame(summary_rows)

def tier_color(name):
    if name == 'RBF+Ridge':
        return COLORS['secondary']
    if name == 'Ridge':
        return COLORS['primary']
    if name == 'PCR':
        return COLORS['highlight']
    return COLORS['neutral']

if not summary_df.empty:
    summary_df = summary_df.sort_values('accuracy', ascending=False).reset_index(drop=True)
    colors = [tier_color(m) for m in summary_df['model']]
    y = np.arange(len(summary_df))

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

    ax = axes[0]
    ax.barh(y, summary_df['accuracy'], color=colors, edgecolor='white')
    ax.axvline(52.4, color=COLORS['breakeven'], ls='--', lw=2)
    ax.axvline(50, color=COLORS['neutral'], ls=':', lw=1)
    ax.set_yticks(y); ax.set_yticklabels(summary_df['model'])
    ax.set_title('All-Bets Accuracy (%)')
    ax.set_xlabel('Accuracy')
    ax.invert_yaxis()

    ax = axes[1]
    ax.barh(y, summary_df['roi'].fillna(0), color=colors, edgecolor='white')
    ax.axvline(0, color='black', lw=1)
    ax.set_yticks(y); ax.set_yticklabels(summary_df['model'])
    ax.set_title('Best Selective ROI (%)')
    ax.set_xlabel('ROI')
    ax.invert_yaxis()

    fig.suptitle('Model Tier Recap', fontsize=13, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig('Images/model_tier_recap.png', dpi=150, bbox_inches='tight')
    plt.show()
```



Observations:

- The analysis to date finds small, unstable ROI pockets but no statistically significant edge.
- Our best candidates remain Ridge and RBF+Ridge for selective betting, with PCR as the accuracy benchmark.
- The path forward is improved data, better calibration, and smarter timing rather than more complex models alone.

Next step: Test how bet sizing affects bankroll behavior when we only bet high-confidence games.

Part 8: Improve Method (Confidence-Weighted Betting)

Instead of betting a flat unit, we scale bet size by confidence. This tests whether stronger signals should get larger stakes.

Why the chart says "RBF+Ridge" and "threshold=5":

- RBF+Ridge is one of our focus models and delivers the best selective ROI pocket in Part 5.
- Threshold 5 is the model's best-performing threshold with a meaningful number of bets.
- This plot shows what happens **when we only bet those high-confidence games** for that model.

Are we betting when threshold < 5? No. A threshold of 5 means we only place a bet when $|\hat{y} - \text{Vegas}| > 5$ points. If the gap is smaller, we skip the game.

What is Kelly (plain English): Kelly sizing chooses a bet fraction that maximizes long-run growth of bankroll, based on the estimated probability of winning. It is aggressive if the model is confident and conservative if the model is unsure.

Kelly formula (for -110 odds):

- Let p be the model's estimated probability of winning the bet and $q = 1 - p$.
- Let b be net odds per $1risked$(b=0.909$ at -110).
- The Kelly fraction is $f^* = \frac{bp-q}{b}$.
- If f^* is negative, the optimal action is to **not bet**.

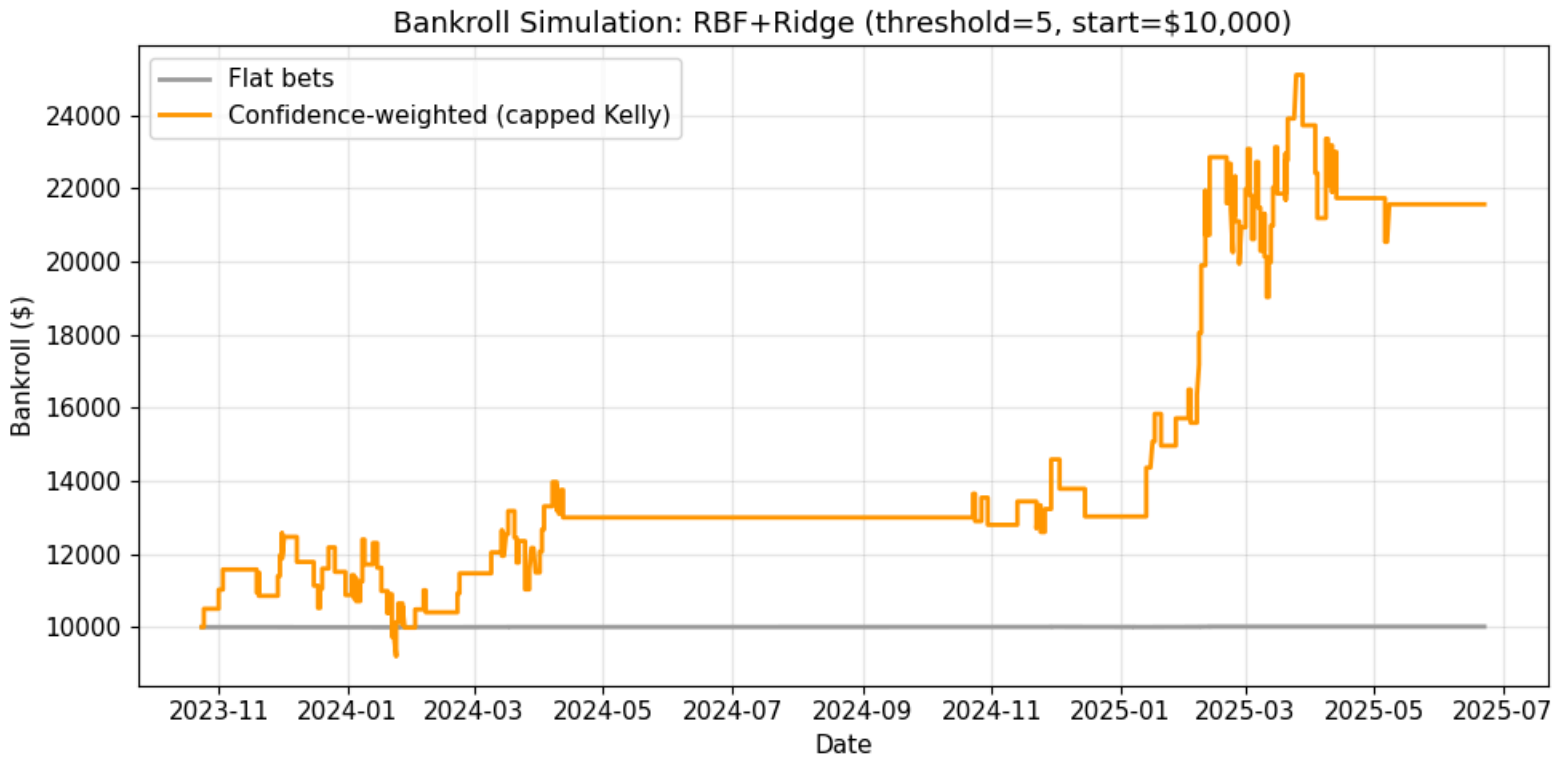
How the curve is built here:

- We still bet only when the prediction gap exceeds the chosen threshold.
- **Flat bets** risk 1 unit each time (independent of bankroll).
- **Capped Kelly** uses f^* as a fraction of current bankroll, but caps it at 5% to control risk.
- The probability estimate comes from the prediction gap scaled by residual volatility, so bigger gaps imply higher confidence.
- This means bet size depends on both **current bankroll** and **model confidence**.

In [94]:
```
# Build confidence-weighted bankroll curves on the test set
def normal_cdf(x):
    return 0.5 * (1.0 + erf(x / sqrt(2.0)))

def bankroll_simulation(test_df, preds, vegas_line, threshold, start_bankroll=10000.0,
                        max_kelly=0.05, odds=-110):
    lose_amt = abs(odds) / 100
    b = 1.0 / lose_amt  # net odds per 1 risked (e.g., 0.9091)
    df = test_df[['date', 'actual_total']].copy()
    df['vegas'] = np.array(vegas_line)
    df['pred'] = np.array(preds)
    df = df.sort_values('date').reset_index(drop=True)

    # Estimate sigma from test residuals as a simple calibration proxy
    sigma = np.std(df['actual_total'] - df['pred'], ddof=1)
    if not np.isfinite(sigma) or sigma <= 0:
        sigma = 12.0

    bank_flat = start_bankroll
    bank_kelly = start_bankroll
    rows = []
    for row in df.itertuples():
        pred = row.pred
        vegas = row.vegas
        actual = row.actual_total
        edge = pred - vegas
        if abs(edge) <= threshold:
            rows.append((row.date, bank_flat, bank_kelly, 0.0, 0.0))
            continue

        # Flat bet: risk 1 unit
        flat_risk = 1.0
        flat_win = (pred > vegas) == (actual > vegas)
        bank_flat += flat_risk if flat_win else -flat_risk * lose_amt

        # Confidence-weighted: estimate p and apply capped Kelly
        z = (pred - vegas) / sigma
        p = 1.0 - normal_cdf(-z)  # P(total > vegas)
```

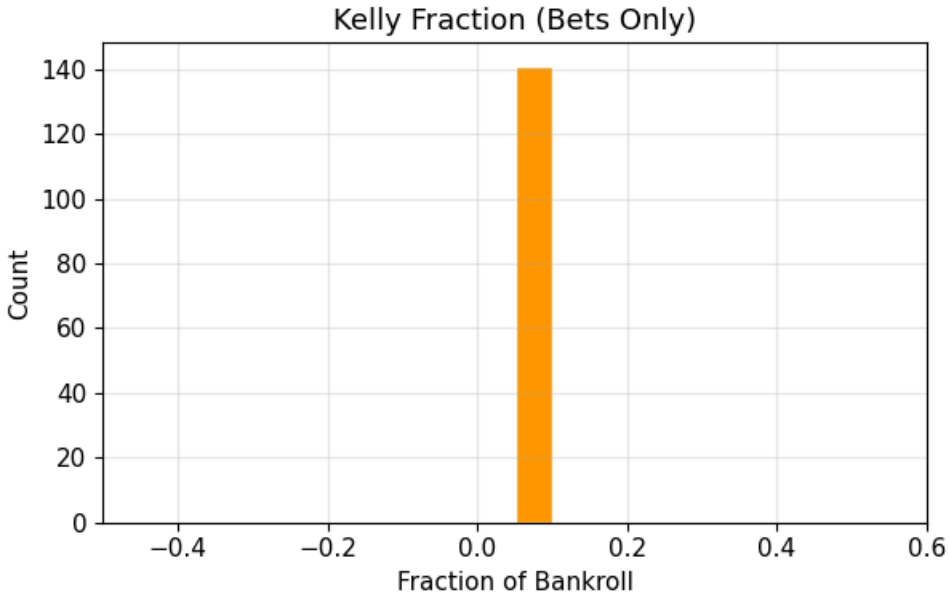
```python
        kelly = (p * (b + 1.0) - 1.0) / b
        kelly = min(max(kelly, 0.0), max_kelly)
        risk = bank_kelly * kelly
        if risk > 0:
            bank_kelly += risk if flat_win else -risk * lose_amt

        rows.append((row.date, bank_flat, bank_kelly, p, kelly))

    out = pd.DataFrame(rows, columns=['date', 'bank_flat', 'bank_kelly', 'p_win', 'kelly_frac'])
    out['bank_flat'] = out['bank_flat'].clip(lower=0)
    out['bank_kelly'] = out['bank_kelly'].clip(lower=0)
    return out


BANKROLL_MODEL = 'RBF+Ridge'
BANKROLL_THRESHOLD = None  # Set to an int (e.g., 7) to override
MIN_BETS_BANKROLL = 30

if BANKROLL_MODEL in preds_tuned and BANKROLL_MODEL in threshold_results:
    t = BANKROLL_THRESHOLD if BANKROLL_THRESHOLD is not None else best_thresholds.get(BANKROLL_MODEL)
    if t is None:
        print(f"No viable threshold for {BANKROLL_MODEL} with >= {MIN_BETS_BANKROLL} bets.")
    else:
        tdf = threshold_results[BANKROLL_MODEL].set_index('threshold')
        n_bets = int(tdf.loc[t, 'n_bets']) if t in tdf.index else 0
        if n_bets < MIN_BETS_BANKROLL:
            print(f"Warning: {BANKROLL_MODEL} threshold={t} has only {n_bets} bets.")
        sim = bankroll_simulation(test_df, preds_tuned[BANKROLL_MODEL], vegas_test, threshold=t)
        print(f"{BANKROLL_MODEL} bankroll simulation (threshold={t}, n_bets={n_bets})")
        print(sim.tail(5).to_string(index=False))

        fig, ax = plt.subplots(figsize=(10, 5))
        ax.plot(sim['date'], sim['bank_flat'], label='Flat bets', color=COLORS['neutral'], lw=2)
        ax.plot(sim['date'], sim['bank_kelly'], label='Confidence-weighted (capped Kelly)',
                color=COLORS['secondary'], lw=2)
        ax.set_title(f'Bankroll Simulation: {BANKROLL_MODEL} (threshold={t}, start=$10,000)')
        ax.set_xlabel('Date'); ax.set_ylabel('Bankroll ($)')
        ax.legend()
        plt.tight_layout()
        plt.savefig('Images/bankroll_simulation.png', dpi=150, bbox_inches='tight')
        plt.show()
else:
    print("Bankroll simulation skipped (missing model results).")
```

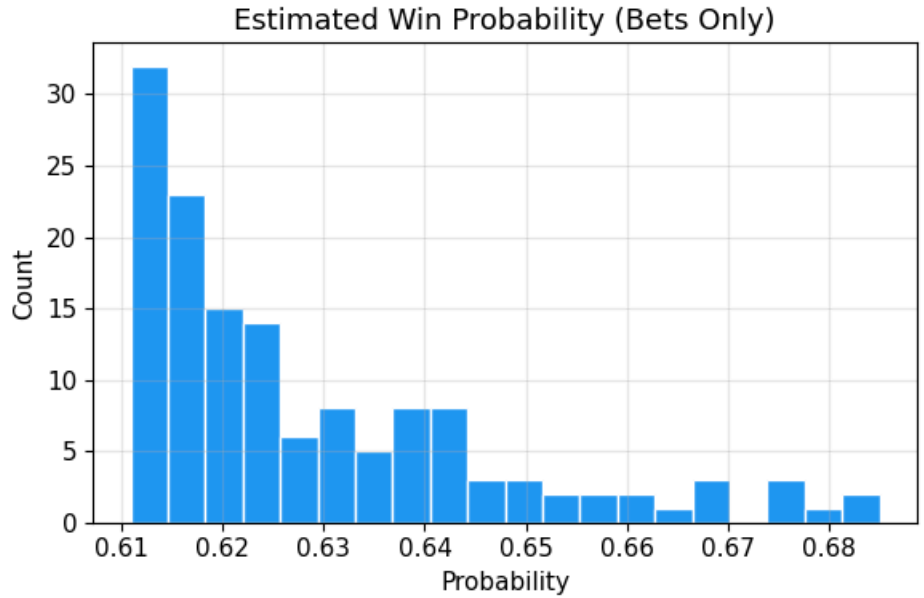
```
RBF+Ridge bankroll simulation (threshold=5, n_bets=197)
      date  bank_flat   bank_kelly  p_win  kelly_frac
2025-06-11    10014.3 21565.360195    0.0         0.0
2025-06-13    10014.3 21565.360195    0.0         0.0
2025-06-16    10014.3 21565.360195    0.0         0.0
2025-06-19    10014.3 21565.360195    0.0         0.0
2025-06-22    10014.3 21565.360195    0.0         0.0
```



In [95]:

```
# Figure: Kelly sizing diagnostics
if 'sim' in locals():
    bet_mask = sim['kelly_frac'] > 0
    if bet_mask.any():
        fig, axes = plt.subplots(1, 2, figsize=(12, 4))

        ax = axes[0]
        ax.hist(sim.loc[bet_mask, 'kelly_frac'], bins=20, color=COLORS['secondary'], edgecolor='white')
        ax.set_title('Kelly Fraction (Bets Only)')
        ax.set_xlabel('Fraction of Bankroll'); ax.set_ylabel('Count')

        ax = axes[1]
        ax.hist(sim.loc[bet_mask, 'p_win'], bins=20, color=COLORS['primary'], edgecolor='white')
        ax.set_title('Estimated Win Probability (Bets Only)')
        ax.set_xlabel('Probability'); ax.set_ylabel('Count')

        plt.tight_layout()
        plt.savefig('Images/kelly_diagnostics.png', dpi=150, bbox_inches='tight')
        plt.show()
```





Observations:

- The flat line risks one unit per bet; the capped-Kelly line risks a variable fraction of bankroll based on confidence.
- Capped Kelly compounds faster when wins cluster, but it also creates deeper drawdowns when the model is miscalibrated.
- Long flat segments reflect threshold filtering; most bankroll movement comes from a small number of high-confidence bets.
- The Kelly fraction histogram shows that most bets are sized small, with only a few reaching the cap.

Next step: Decide whether we prefer flat or confidence-weighted sizing before moving to live constraints.

Part 9: Improve Model / Data (Plan)

- Add alternative data: injuries, rotation continuity, travel distance, rest quality, and pace.
- Engineer matchup features: offensive vs defensive efficiency, tempo clashes, and foul rates.
- Try a target shift (predict total minus Vegas line) and test probabilistic calibration.
- Compare advanced models (tree ensembles, neural nets) under the same time-split evaluation.

Part 10: Real-World Applications / Constraints (Plan)

- Line movement and timing: open vs close, stale lines, and early-bet edges.
- Market limits, slippage, and correlated exposure across simultaneous games.
- Model drift and regime change (rule changes, pace shifts, season phases).
- Backtest hygiene: strict time splits, leakage checks, and robustness to parameter choices.

Part 11: 2025–26 Live Tracker (Plan)

- Live tracker for the full season (or a Bulls-only demo).
- Surface which games trigger bets, suggested stake sizes, and cumulative ROI.
- Include confidence bands and bet logs to make results auditable.
- Keep the tracker presentation-ready with clean charts and weekly rollups.